

Ygg: Tree-Based Collaborative Speculative Decoding with Token-Only Transmission

Sijia Li^{†‡}, Yumeng Liang[‡], Ruiqi Li[§], Jianjiang Li^{†*}, and Jie Wu^{‡¶*}

[†]University of Science and Technology Beijing, Beijing, China

[‡]China Telecom Cloud Computing Research Institute, Beijing, China

[§]Peking University, Beijing, China

[¶]Temple University, USA

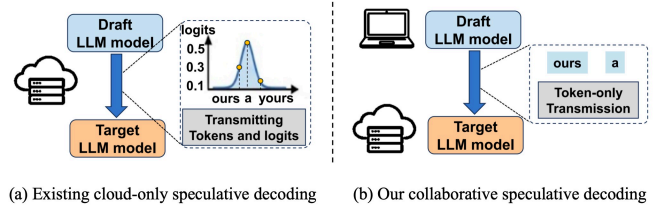
lisijia@xs.ustb.edu.cn, liangym9@chinatelecom.cn, rqli25@stu.pku.edu.cn, lijianjiang@ustb.edu.cn, jiewu@temple.edu

Abstract—While Large Language Models (LLMs) have exhibited remarkable performance, the QoS for the LLM inference system is still constrained by the substantial latency inherent in autoregressive decoding. To accelerate the decoding process, speculative decoding has recently been proposed, employing both a lightweight draft model and the target LLM residing in the cloud to rapidly generate candidate tokens and verify these candidates in parallel, enabling the processing of multiple tokens per step. However, co-deploying the draft and target models in the cloud wastes valuable GPU memory resources and limits the cloud’s ability to perform parallel inferences for higher throughput. To further improve inference QoS by achieving higher throughput, in this paper, we propose Ygg to offload the draft model onto the cost-effective edge device, relieving the computational workload of the cloud in collaborative manners. To address the drawback of excessive data transmission in existing speculative decoding methods, we design a novel collaborative paradigm with token-only transmission, which drastically cuts down communication latency. Furthermore, Ygg incorporates two additional key designs: budget-aware adaptive tree-based draft generation and asynchronous pipelining, which serve to elevate the acceptance rate of generated draft tokens and boost inference throughput, respectively. Experimental results demonstrate that our proposed framework achieves up to 1.94× throughput improvement and reduces serving costs by 47.3% compared to the cloud-only speculative decoding method. Furthermore, in comparison with the state-of-the-art edge-cloud collaborative method, Ygg reduces the overall inference latency by up to 49.3% and consistently achieves the best performance across diverse tasks and varying LLM scales.

Index Terms—LLM, Collaborative Inference, Speculative Decoding, QoS of LLM inference

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated remarkable capabilities in reasoning and text generation [1], [2], but the QoS for the LLM inference system is still constrained by the significant computational and latency burdens of sequential auto-regressive decoding on cloud infrastructure [3]. To accelerate the decoding process so as to improve the QoS of LLMs inference, *speculative decoding* has recently been



(a) Existing cloud-only speculative decoding (b) Our collaborative speculative decoding

Fig. 1: Illustration of prior speculative decoding methods and ours. Prior methods deploy both draft and target LLMs on the cloud to generate candidate tokens and verify them. Whereas we offload the draft model to an inexpensive edge device to alleviate the cloud GPU burden and design a token-only transmission mechanism to reduce the communication latency.

proposed [4]–[6], in which both the lightweight draft model and the target LLM are deployed on the cloud. The draft model rapidly generates candidate tokens, while the target model verifies these candidates in parallel, enabling multiple tokens to be processed in a single step. Although such a draft-target model collaboration achieves significant acceleration gains, *the additional deployment of the draft model on the cloud consumes valuable cloud GPU memory*, which restricts large-scale parallel processing and thereby limits further improvement of inference throughput.

Luckily, the computing power of edge GPUs has been growing steadily, which is highly cost-effective compared with cloud servers [7], creating a promising opportunity to alleviate the cloud-side inference burden [8]. Given the draft model’s moderate demands of computation resources and GPU memory, in this paper, we envision *whether we can offload the lightweight draft model to inexpensive edge devices, and improve the inference QoS by achieving higher throughput in an edge-cloud collaborative manner*. In this way, the valuable GPU memory on the cloud could be saved for the target LLMs to process more samples in parallel, so as to improve the overall throughput.

However, collaborative speculative decoding faces following unique challenges in achieving satisfactory acceleration:

(i) *Foremost, in conventional speculative decoding, the large volume of data transmitted between the draft and target*

Sijia Li and Yumeng Liang contributed equally to this work.

*: Corresponding authors.

Ygg: 基于树的协作投机解码与仅令牌传输

李思佳^{†‡}、梁雨萌[‡]、李瑞奇[§]、李建江^{†*} 和 吴杰^{‡¶*} [†]北京科技大学, 中国北京 [‡]中国电信云计算研究院, 中国北京 [§]北京大学, 中国北京 [¶]美国天普大学 lisijia@xs.ustb.edu.cn, liangym9@chinatelecom.cn, rqli25@stu.pku.edu.cn, lijianjiang@ustb.edu.cn, jiewu@temple.edu

Abstract——尽管大型语言模型（LLMs）表现出了卓越的性能，LLM推理系统的服务质量（QoS）仍受限于自回归解码固有的较高延迟。为了加速解码过程，最近提出了推测性解码方法，该方法同时使用轻量级草稿模型和位于云端的目标LLM，快速生成候选标记并并行验证这些候选，从而实现每步处理多个标记。然而，将草稿模型和目标模型共同部署在云端会浪费宝贵的GPU内存资源，并限制云端执行并行推理以提升吞吐量的能力。为了通过实现更高吞吐量进一步提高推理QoS，本文提出了Ygg，将草稿模型卸载到成本更低的边缘设备上，以协作方式减轻云端的计算负载。为了解决现有推测性解码中过度数据传输的缺点

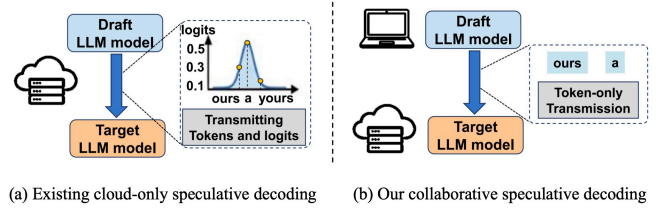


图 1: 先前的推测解码方法与我们的方法示意图。先前的方法在云端部署草稿和目标大语言模型生成候选标记并进行验证。而我们将草稿模型卸载到廉价的边缘设备上，以减轻云端 GPU 负担，并设计了仅传输标记的机制以降低通信延迟。

Index Terms——大型语言模型，协作推理，投机解码，大型语言模型推理的服务质量

一、引言

大型语言模型（LLMs）在推理和文本生成方面表现出了显著的能力 [1], [2]，但LLM推理系统的服务质量（QoS）仍然受到云基础设施上顺序自回归解码所带来的巨大计算和延迟负担的限制 [3]。为了加速解码过程，从而提高LLMs推理的服务质量，speculative decoding 最近被

提出的 [4]–[6]，其中轻量级草稿模型和目标大型语言模型（LLM）都部署在云端。草稿模型快速生成候选标记，而目标模型并行验证这些候选标记，使得多个标记可以在单步中处理。尽管这种草稿-目标模型的协作实现了显著的加速效果，the additional deployment of the draft model on the cloud consumes valuable cloud GPU memory, 这限制了大规模并行处理，从而限制了推理吞吐量的进一步提高。

幸运的是，边缘 GPU 的计算能力一直在稳步增长，与云服务器相比，这具有很高的性价比 [7]，从而创造了一个有前景的机会来减轻云端推理的负担 [8]。鉴于草稿模型对计算资源和 GPU 内存的适度需求，本文设想 *whether we can offload the lightweight draft model to inexpensive edge devices, and improve the inference QoS by achieving higher throughput in an edge-cloud collaborative manner*。通过这种方式，可以将云端宝贵的 GPU 内存节省出来，让目标大型语言模型能够并行处理更多样本，从而提高整体吞吐量。

然而，协作式推测解码在实现令人满意的加速时面临以下独特挑战：

(i) *Foremost, in conventional speculative decoding, the large volume of data transmitted between the draft and target*

Sijia Li and Yumeng Liang contributed equally to this work.
*: Corresponding authors.

models introduces substantial communication latency. The candidate tokens generated by the draft model, as well as their associated probability distributions (logits), are required to be transmitted to the target LLM model. The high-dimensional logits for every drafted token incurs substantial communication latency overhead, which even outweighs the performance gains brought by the collaboration itself. Although some existing methods [9], [10] attempt to alleviate this issue by splitting the verification phase or transmitting only top-k sparse logits, the high-dimensional logits remains a critical bottleneck for communication latency. *In this paper, we rethink the form of speculation itself, and employ a token-only mechanism to perform speculative decoding without transmitting logits.* As illustrated in Fig. 1, our token-only communication significantly reduces transmission overhead, thereby achieving higher throughput gains.

(ii) *Although transmitting fewer tokens can reduce communication latency, the resulting drop in the acceptance rate of candidate tokens degrades the overall acceleration performance.* The acceleration performance of speculative decoding depends on the acceptance probability of the candidate tokens generated by the draft model during the verification of the target LLM, *i.e.*, any rejected token must be regenerated, which directly impairs the end-to-end speedup. When transmitting fewer candidate tokens, the hit rate of the draft model will drop, leading to more frequent token regeneration. Thus, striking a balance between communication overhead and acceptance rate to achieve stronger overall acceleration becomes a critical problem. To tackle this challenge, we regard the generation of sequential candidate tokens as the construction of a tree structure and propose the *budget-aware adaptive tree-based draft generation scheme*. Instead of aggressively generating a large number of redundant nodes, we dynamically select the highest-confidence nodes from a global buffer and control the total number of nodes using a budget constraint. In this way, we can achieve a high candidate token acceptance rate with a small transmission volume.

(iii) *Collaboration between draft generation on the edge and verification on the cloud tends to introduce bubbles, which limits the improvement of overall inference speed.* In traditional speculative decoding, the draft and target models work in a serial manner, avoiding the issue of excessive cloud memory occupation caused by parallel processing. However, such a concern is nonexistent in our collaboration mode, and the pipeline parallel mode can better utilize the resources of both sides to achieve acceleration. Nevertheless, resource disparities and model differences between the edge and cloud during parallel processing cause inconsistent execution speeds, introducing bubbles. To address this, we design an *asynchronous pipelining* method that adjusts the number of draft tokens based on cloud verification latency, fully utilizing resources and achieving higher acceleration.

The corresponding methods to address the above challenges

are integrated into a framework named Ygg¹, which enables tree-based collaborative speculative decoding with token-only transmission. We implemented Ygg in the real edge-cloud testbed and evaluate its performance via extensive experiments. Experimental results show that Ygg achieves a $1.94\times$ throughput gain compared with the cloud-side speculative decoding method, and reduce the cost by 47.3% by leveraging an inexpensive edge device. In comparison with other state-of-the-art edge-cloud collaborative speculative decoding methods, our approach cuts the inference latency by 49.3%, which validates the effectiveness of the proposed method.

Our contributions are summarized as follows:

- We propose a communication-efficient edge–cloud speculative decoding framework Ygg to improve the LLMs inference QoS. It eliminates the transmission of draft logits by introducing token-only, tree-structured speculation.
- A budget-aware adaptive tree-based draft generation scheme is proposed to improve acceptance rates of candidate tokens under tight token budgets. And an asynchronous pipelining mechanism is designed to improve inference throughput.
- Extensive experiments on the real edge-cloud testbed validate that Ygg can achieve up to $1.94\times$ speedup of throughput and outperforms the other existing collaborative speculative decoding methods.

II. BACKGROUND AND MOTIVATION

A. Speculative Decoding

LLMs generate text autoregressively, which is often bounded by memory rather than arithmetic intensity [11], [12]. To accelerate this, speculative decoding [4]–[6] introduces a lightweight draft model M_q to generate γ candidate tokens, which are then verified in parallel by the target model M_p . In this way, the target model can leverage multiple candidate tokens rapidly generated by the lightweight model to batch-process and generate tokens for multiple positions in one step.

The core mechanism of speculative decoding is how to verify the candidate tokens generated by the draft model. Let $P_i(x)$ and $Q_i(x)$ denote the probability distributions of the target and draft models at the i -th step, respectively. The verification process consists of two key phases:

a) Accept/Reject: For a drafted candidate token x_i , let $p_i(x_i)$ and $q_i(x_i)$ denote probability values within $P_i(x)$ and $Q_i(x)$. The target model evaluates the acceptance probability:

$$\Pr(\text{accept } x_i) = \min(1, p_i(x_i)/q_i(x_i)). \quad (1)$$

If accepted, the token is retained. Otherwise, it is rejected, and the process moves to the resampling phase.

b) Resample: To ensure the output strictly follows the target distribution, a rejected token is replaced by a resampled new token x'_i from the rectified distribution:

$$x'_i \sim \text{norm}(\max(0, P_i(x) - Q_i(x))). \quad (2)$$

¹Named after Yggdrasil, the world tree in Norse mythology that connects realms of different worlds.

models introduces substantial communication latency. 由草稿模型生成的候选标记及其相关的概率分布 (logits) 需要传输到目标大型语言模型 (LLM)。每个草稿标记的高维 logits 会带来巨大的通信延迟开销, 这甚至超过了协作本身带来的性能提升。尽管一些现有方法[9], [10]试图通过拆分验证阶段或仅传输 top-k 稀疏 logits 来缓解此问题, 高维 logits 仍然是通信延迟的关键瓶颈。如图1所示, 我们的仅标记通信显著减少了传输开销, 从而实现更高的吞吐量提升。

(ii) *Although transmitting fewer tokens can reduce communication latency, the resulting drop in the acceptance rate of candidate tokens degrades the overall acceleration performance.* 投机性解码的加速性能取决于在目标大语言模型 (LLM) i.e. 验证过程中, 草稿模型生成的候选标记的接受概率, 任何被拒绝的标记都必须重新生成, 这会直接影响端到端的加速。当传输较少的候选标记时, 草稿模型的命中率会下降, 导致标记重新生成的频率增加。因此, 在通信开销和接受率之间取得平衡以实现更强的整体加速成为一个关键问题。为了解决这一挑战, 我们将顺序候选标记的生成视为树结构的构建, 并提出了 *budget-aware adaptive tree-based draft generation scheme*。我们不是积极生成大量冗余节点, 而是从全局缓冲区动态选择置信度最高的节点, 并通过预算约束控制节点总数。通过这种方式, 我们可以在传输量较小的情况下实现高候选标记接受率。

(iii) *Collaboration between draft generation on the edge and verification on the cloud tends to introduce bubbles, which limits the improvement of overall inference speed.* 在传统的推测解码中, 草稿模型和目标模型以串行方式工作, 从而避免了并行处理导致的过度云端内存占用问题。然而, 在我们的协作模式下, 这种问题不存在, 并且流水线并行模式可以更好地利用双方的资源以实现加速。然而, 边缘端与云端在并行处理过程中存在资源差异和模型差异, 导致执行速度不一致, 产生空闲周期。为了解决这一问题, 我们设计了一种 *asynchronous pipelining* 方法, 根据云端验证延迟调整草稿令牌的数量, 充分利用资源并实现更高的加速。

应对上述挑战的相应方法

集成到名为 Ygg^1 的框架中, 该框架支持基于树的协作式推测解码, 并仅传输令牌。我们在实际边缘云测试床中实现了 Ygg , 并通过大量实验评估其性能。实验结果表明, 与云端推测解码方法相比, Ygg 实现了 $1.94\times$ 的吞吐量提升, 并通过利用廉价的边缘设备将成本降低了 47.3%。与其他最先进的边缘云协作推测解码方法相比, 我们的方法将推理延迟降低了 49.3%, 验证了所提方法的有效性。

我们的贡献总结如下:

- 我们提出了一个通信高效的边缘-云投机解码框架 Ygg , 以提高大型语言模型的推理服务质量。它通过引入仅令牌的树状结构投机, 消除了草稿 logits 的传输。
- 提出了一种预算感知的自适应基于树的草稿生成方案, 以在严格的令牌预算下提高候选令牌的接受率。并设计了一种异步流水线机制以提高推理吞吐量。
- 在真实的边缘云测试平台上进行的大量实验验证了 Ygg 可以将吞吐量提升最多 $1.94\times$ 倍, 并且优于其他现有的协作推测解码方法。

II. 背景与动机

A. Speculative Decoding

大型语言模型 (LLMs) 以自回归方式生成文本, 这通常受到内存限制而非算术密集度的约束 [11], [12]。为加速这一过程, 投机解码 [4]–[6] 引入了一个轻量级草稿模型 M_q 来生成 γ 个候选标记, 然后由目标模型 M_p 并行验证。通过这种方式, 目标模型可以利用轻量级模型快速生成的多个候选标记, 批处理并在一步中生成多个位置的标记。

投机解码的核心机制是如何验证草稿模型生成的候选标记。令 $P_i(x)$ 和 $Q_i(x)$ 分别表示目标模型和草稿模型在第 i 步的概率分布。验证过程包括两个关键阶段:

a) 接受/拒绝: 对于一个草拟的候选标记 x_i , 令 $p_i(x_i)$ 和 $q_i(x_i)$ 表示 $P_i(x)$ 与 $Q_i(x)$ 内的概率值。目标模型评估接受概率:

$$\Pr(\text{accept } x_i) = \min(1, p_i(x_i)/q_i(x_i)). \quad (1)$$

如果被接受, 该令牌将被保留。否则, 它将被拒绝, 流程将转到重新采样阶段。

b) 重新采样: 为了确保输出严格遵循目标分布, 被拒绝的令牌将被来自修正分布的重新采样新令牌 x'_i 替换:

$$x'_i \sim \text{norm}(\max(0, P_i(x) - Q_i(x))). \quad (2)$$

¹以北欧神话中的世界树尤克德拉希命名, 它连接着不同世界的领域。

This mechanism guarantees the identical distribution as that of the target model M_p , as long as it has access to the full conditional distribution $Q_i(x)$ of the approximation. Therefore, in traditional speculative decoding methods, the candidate tokens as well as the corresponding logits are sent to the target LLMs model, as shown in Fig. 1 (a). This mechanism incurs little latency within the cloud, yet becomes a communication bottleneck in edge-cloud collaborative inference.

B. The Communication Bottleneck in Edge-Cloud Collaborative Speculative Decoding

In an edge-cloud collaborative setting, placing the draft model on the edge and the target model on the cloud is a promising architecture [13], [14]. However, applying standard speculative decoding creates a communication bottleneck. Calculating Eq. (2) necessitates transmitting the probability distribution for every draft token. For a vocabulary size V (typically 32k–128k) and draft length N , this results in a transmission payload of $N \times V$, often reaching 1MB per step. While this data transfer overhead is negligible in standard SD deployments within high-bandwidth cloud clusters (*e.g.*, via NVLink), it becomes a prohibitive bottleneck in edge-cloud scenarios operating over wireless networks [15]. Although recent optimization strategies such as DSSD [9] and Top-K Sparse Logits Transmission (TK-SLT) [10] attempt to mitigate this overhead by splitting verification phases or transmitting sparse logits, they do not avoid transmitting probability distributions. In this paper, we rethink the form of speculation and employ a novel speculative decoding mechanism with token-only transmission to reduce the communication overhead.

III. METHODOLOGY

A. Kernel Idea: Naive Sampling and Tree Attention

To ensure the generated output remains distributionally consistent with the target model without accessing draft logits, we employ *Naive Sampling* (NS) [16] for verification. Specifically, the target model M_p independently computes the target distribution $P_i(x)$ and samples a verification token x' accordingly. The draft token x_i is accepted if and only if it strictly matches x' :

$$\text{Accept}(x_i) = \mathbb{I}(x_i = x'), \quad \text{where } x' \sim P_i(x). \quad (3)$$

Although efficient, NS has a low acceptance rate, especially when few candidates are generated for each position, because of its strict matching criterion. Transmitting many candidates causes high communication overhead, so a strategy is needed to balance the acceptance rate and transmission cost.

To recover the acceptance rate under low communication cost, we introduce a *tree-based drafting strategy*. Instead of producing a single sequence, the draft model M_q constructs a token tree $\mathcal{N}$ covering multiple plausible paths. Using tree attention [16]–[18], the target model M_p verifies the entire tree in one forward pass. Specifically, the topology-aware mask $\mathbf{M}$ allows a query token u to attend to a key token v only if v is an ancestor of u :

$$\mathbf{M}_{uv} = \begin{cases} 0, & \text{if } v \in \text{Ancestors}(u) \cup \{u\}; \\ -\infty, & \text{otherwise.} \end{cases} \quad (4)$$

By applying this topology-aware mask, the verification complexity for a tree of size $|\mathcal{N}|$ remains $O(1)$ through parallel computation, ensuring that the latency cost is comparable to verifying a single chain.

We implement the above idea in a well-designed edge-cloud collaborative speculative decoding frame. In the following, we separately introduce the framework overview and the two kernel designs.

B. Framework Overview

This section presents our communication-efficient framework Ygg, designed for edge-cloud collaborative speculative decoding. To overcome the critical communication bottleneck in standard speculative decoding caused by the transmission of high-dimensional probability distributions, we propose an token-only communication protocol. By combining budget-aware adaptive tree construction on the device side with a pipelined draft verification mechanism, the proposed framework enables fast edge–cloud collaborative inference. As illustrated in Fig. 2, the framework operates in two main stages connected by a lightweight protocol:

Edge-Side Draft. The process initiates on an edge device hosting a lightweight draft model. Instead of generating a single sequence, the drafter constructs a token tree to maximize acceptance rate. As shown in the Draft panel, the model outputs token probability distributions $(p_0, p_1, \dots)$ from which multiple candidate tokens are sampled. Crucially, candidates not immediately selected for the current tree expansion are retained in a global candidate buffer pool. These buffered tokens compete in subsequent selection rounds, allowing high-confidence candidates from previous steps to be reconsidered, thereby constructing a high-quality tree topology.

Token-Only Communication. To break the bandwidth bottleneck, we avoid transmitting probability distributions. Instead, the device packages the drafted tree into a lightweight speculation packet $\mathcal{P}$, containing only discrete integer indices:

$$\mathcal{P} = (\mathbf{t}, \mathbf{p}, \mathbf{pos}), \quad (5)$$

where $\mathbf{t}$ represents the token IDs, $\mathbf{p}$ denotes the parent indices defining the tree topology, and $\mathbf{pos}$ indicates the position IDs for alignment.

Cloud-Side Verify. The packet $\mathcal{P}$ is transmitted to the cloud server hosting the large-parameter target model. Upon receipt, the target model constructs a tree attention mask based on $\mathbf{p}$ to compute logits $(q_0, q_1, \dots)$ for all nodes in a single forward pass. Since draft logits are unavailable, we employ NS verification strategy. The cloud performs path-based dynamic verification starting from the root: it samples a target token t' from its computed distribution and compares it against the drafted child node. If t' matches, verification proceeds to the next level; otherwise, the process terminates, and t' is returned

该机制保证了与目标模型 M_p 相同的分布，只要它能够访问近似的完整条件分布 $Q_i(x)$ 。因此，在传统的投机解码方法中，候选标记以及相应的 logits 会被发送到目标 LLM 模型，如图 1 (a) 所示。该机制在云端产生的延迟很小，但在边缘-云协作推理中却成为了通信瓶颈。

B. The Communication Bottleneck in Edge-Cloud Collaborative Speculative Decoding

在边缘-云协作环境中，将草稿模型放置在边缘端而将目标模型放置在云端是一种有前景的架构 [13], [14]。然而，应用标准的推测解码会产生通信瓶颈。计算式 (2) 需要传输每个草稿 token 的概率分布。对于词汇表大小 V (通常为 32k–128k) 和草稿长度 N ，这会导致 $N \times V$ 的传输负载，通常每步可达到 1MB。在高带宽云集群内部署标准 SD 时 (e.g., 通过 NVLink)，这种数据传输开销可以忽略，但在通过无线网络运行的边缘-云场景中，这成为了一个难以逾越的瓶颈 [15]。尽管最近的优化策略如 DSSD [9] 和 Top-K 稀疏 logits 传输 (TK-SLT) [10] 尝试通过拆分验证阶段或传输稀疏 logits 来缓解这一开销，但它们并未避免传输概率分布。在本文中，

三、方法论

A. Kernel Idea: Naive Sampling and Tree Attention

为了确保生成的输出在分布上与目标模型保持一致而无需访问草稿日志，我们使用 *Naive Sampling* (NS) [16] 进行验证。具体来说，目标模型 M_p 独立计算目标分布 $P_i(x)$ 并相应地采样一个验证令牌 x' 。只有当草稿令牌 x_i 严格匹配 x' 时，才会被接受：

$$\text{Accept}(x_i) = \mathbb{I}(x_i = x'), \quad \text{where } x' \sim P_i(x). \quad (3)$$

尽管高效，NS 的接受率较低，尤其是在每个职位生成的候选人较少时，因为其严格的匹配标准。传输大量候选人会导致高通信开销，因此需要一种策略来平衡接受率和传输成本。

为了在低通信成本下恢复接受率，我们引入了 *tree-based drafting strategy*。草稿模型 M_q 并不是生成单一序列，而是构建一个涵盖多条可能路径的令牌树 $\mathcal{N}$ 。通过使用树注意力 [16]–[18]，目标模型 M_p 可以在一次前向传播中验证整个树。具体来说，拓扑感知掩码 $\mathbf{M}$ 允许查询令牌 u 仅在 v 是 u 的祖先时才关注键令牌 v ：

$$\mathbf{M}_{uv} = \begin{cases} 0, & \text{if } v \in \text{Ancestors}(u) \cup \{u\}; \\ -\infty, & \text{otherwise.} \end{cases} \quad (4)$$

通过应用这种拓扑感知掩码，大小为 $|\mathcal{N}|$ 的树的验证复杂度通过并行计算保持为 $O(1)$ ，确保延迟成本可与验证单条链相媲美。

我们在一个精心设计的边缘-云协作推测解码框架中实现了上述想法。下面，我们将分别介绍框架概览和两个核心设计。

B. Framework Overview

本节介绍了我们为边缘-云协作推测解码设计的通信高效框架 Ygg。为克服标准推测解码中由于高维概率分布传输而导致的关键通信瓶颈，我们提出了一种仅传输令牌的通信协议。通过将设备端的预算感知自适应树构建与流水线式草稿验证机制相结合，该框架实现了快速的边缘-云协作推理。如图 2 所示，该框架主要通过轻量级协议连接两个主要阶段运行：

边缘端草稿。该过程在托管轻量级草稿模型的边缘设备上启动。草稿器不是生成单一序列，而是构建一个标记树以最大化接受率。如草稿面板所示，模型输出标记概率分布 $(p_0, p_1, \dots)$ ，从中抽取多个候选标记。关键是，未立即被选用于当前树扩展的候选标记会保留在全局候选缓冲池中。这些缓冲标记在随后的选择轮中竞争，使得前一步中的高置信度候选可以被重新考虑，从而构建高质量的树拓扑。

仅令牌通信。为了打破带宽瓶颈，我们避免传输概率分布。相反，设备将草拟的树打包成轻量级推测数据包 $\mathcal{P}$ ，仅包含离散整数索引：

$$\mathcal{P} = (\mathbf{t}, \mathbf{p}, \mathbf{pos}), \quad (5)$$

其中 $\mathbf{t}$ 表示标记 ID， $\mathbf{p}$ 表示定义树拓扑的父索引， $\mathbf{pos}$ 表示用于对齐的位置 ID。

云端验证。数据包 $\mathcal{P}$ 被传输到托管大参数目标模型的云服务器。收到后，目标模型基于 $\mathbf{p}$ 构建一个树注意力掩码，以在单次前向传播中计算所有节点的 logits $(q_0, q_1, \dots)$ 。由于草稿 logits 不可用，我们采用 NS 验证策略。云端从根节点开始执行基于路径的动态验证：它从计算的分布中采样目标 token t' ，并将其与草稿子节点进行比较。如果 t' 匹配，验证继续到下一层；否则，过程终止，并返回 t' 。

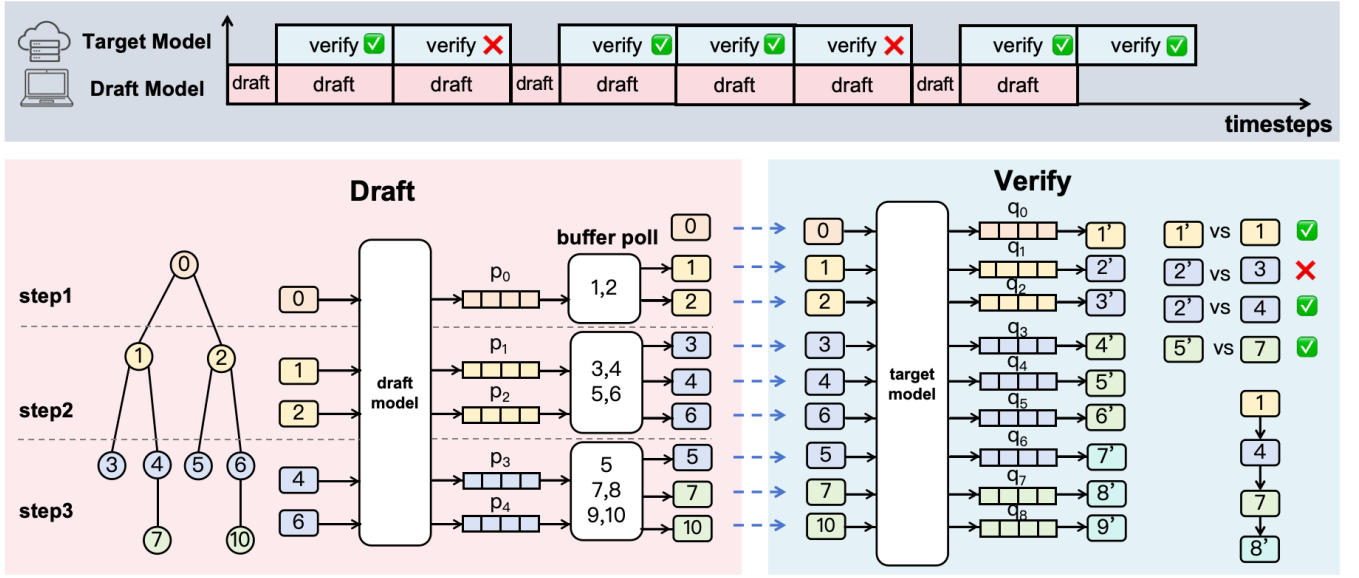


Fig. 2: Overview of Ygg, which adopts edge-cloud collaborative speculative decoding via token-only transmission. The draft model sequentially generate tokens for different positions in multiple steps (e.g., yellow nodes 1–2, blue nodes 3–6). At each step, multiple candidates are produced; the top- k ones with higher confidence scores are selected dynamically, while others remain in a global buffer pool. Only the selected tokens are finally transmitted to the cloud without logits, which are compared with the target LLMs’s output (e.g., target 2’ rejects draft 3, while 5’ accepts 7) in parallel. The top timeline illustrates Asynchronous Pipelining, where the edge pre-drafts future tokens during cloud verification to mask latency bubbles.

as a corrective bonus token. The cloud then sends back a response $\mathcal{R}$ containing the verified path.

As depicted in the top timeline, the framework employs an asynchronous execution model. During the cloud-side verification phase, the device does not idle; instead, it concurrently executes a draft phase for the next round. If the verified path from the cloud matches the prefix of this concurrent draft, the waiting period draft is immediately activated and transmitted as the new request, effectively masking the drafting latency.

C. Budget-Aware Adaptive Tree Construction

To compensate for the reduced acceptance rate of NS, the device must construct a high-quality token tree $\mathcal{T}$ that covers the most probable outputs of the target model. We propose a *Budget-Aware Adaptive Tree Construction* algorithm. Unlike existing approaches that either lack lateral expansion [19] or introduce excessive computational redundancy [20], our approach utilizes normalized probabilities as confidence scores and introduces a global candidate buffer to enable cross-layer competition, addressing the inefficiencies of redundancy and the lack of lateral expansion in existing methods.

Problem Formulation. Prior research demonstrates a significant positive correlation between the output probability of the draft model and the probability of acceptance by the target model. Consequently [20], we utilize the predictive probability of the draft model as a confidence proxy to estimate the likelihood of a generated token being ultimately accepted. Let $\mathcal{V}$ denote the vocabulary. Given a prefix context sequence

$x_{1:t}$, the draft model M_d predicts the next token probability distribution via a softmax operation:

$$P(x_{t+1}|x_{1:t}) = \text{Softmax}(\text{Logits}(M_d(x_{1:t}))). \quad (6)$$

We define the *Token Tree* $\mathcal{T}$ as a set of nodes, where each node u represents a token sequence $x_{1:t+d}$ extending from the root. Considering the sequential dependence inherent in the verification process, where the acceptance of node u necessitates the acceptance of all its ancestor nodes, the probability of the path being accepted by the target model approximates the joint probability of all tokens along the trajectory. Accordingly, we adopt the Cumulative Log-Probability (CLP) as the metric $S(u)$ to quantify the quality of node u :

$$S(u) = \sum_{k=1}^d \log P(x_{t+k}|x_{1:t+k-1}). \quad (7)$$

Our objective is to construct a tree $\mathcal{T}$ subject to strict memory and computational overhead constraints defined by $|\mathcal{T}| \leq K$, where K represents the token budget. Specifically, the tree is designed to include the set of nodes with the highest $S(u)$ values to maximize the coverage of high-confidence regions within the draft model’s predicted distribution, thereby enhancing the speculation success rate of speculative decoding.

Algorithm. Existing tree construction algorithms typically perform layer-wise expansion, selecting the top- k nodes at depth d to expand to $d+1$ while immediately discarding the remainder. We identify two critical limitations of this paradigm within the context of edge-cloud collaborative inference. First, the strategy of generating a large tree before pruning to locate the optimal path introduces excessive computational

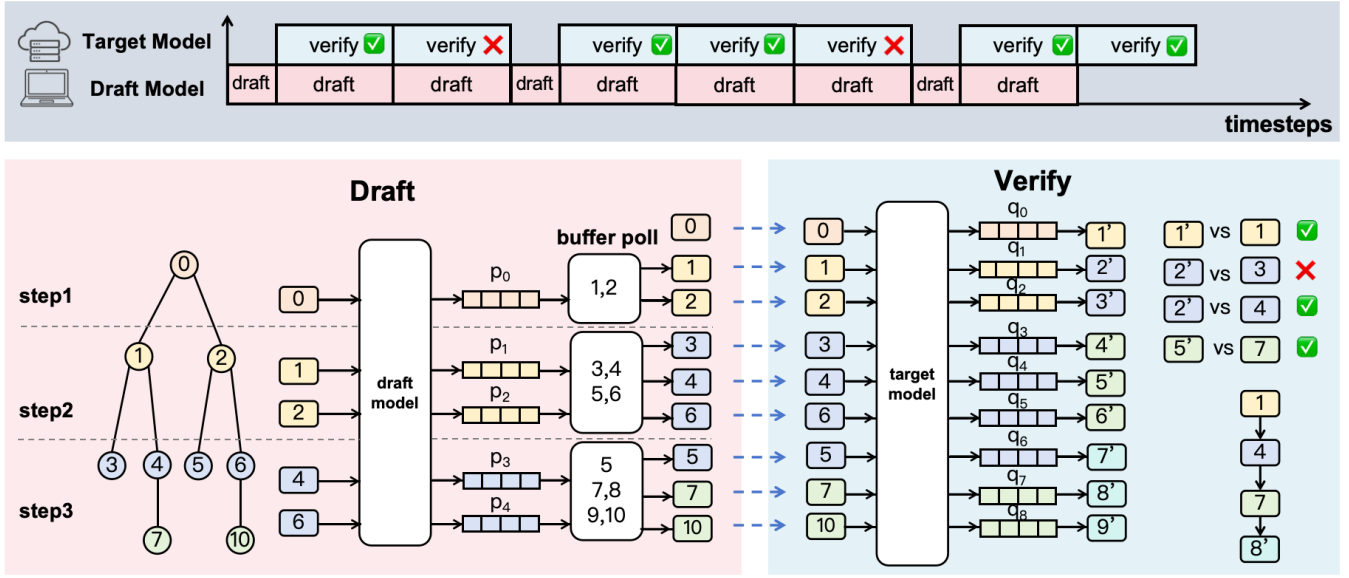


图2: Ygg概览, Ygg通过仅传输令牌的边缘-云协作推测解码。草稿模型在多个步骤中顺序生成不同位置的令牌 (e.g., 黄色节点1-2, 蓝色节点3-6)。在每一步, 会生成多个候选项; 具有较高置信分数的前 k 个会被动态选择, 而其他的保持在全局缓冲池中。只有被选中的令牌最终会被传输到云端, 不带logits, 并与目标LLM的输出 (e.g., 目标2'拒绝草稿3, 而5'接受7) 并行比较。顶部时间轴展示了异步流水线, 其中边缘端在云端验证期间预生成未来令牌, 以掩盖延迟空隙。

作为一个纠正奖励代币。然后云端发送回一个包含已验证路径的响应 $\mathcal{R}$ 。

如顶部时间轴所示, 该框架采用异步执行模型。在云端验证阶段期间, 设备不会闲置; 相反, 它会同时执行下一轮的草稿阶段。如果来自云端的已验证路径与此并行草稿的前缀匹配, 则等待期间的草稿会立即被激活并作为新请求发送, 从而有效掩盖草稿延迟。

C. Budget-Aware Adaptive Tree Construction

为了弥补 NS 接受率下降的问题, 设备必须构建一个高质量的 token 树 $\mathcal{T}$, 覆盖目标模型最可能的输出。我们提出了一种 *Budget-Aware Adaptive Tree Construction* 算法。与现有的方法不同, 现有方法要么缺乏横向扩展 [19], 要么引入过多的计算冗余 [20], 我们的方法利用归一化概率作为置信度分数, 并引入全局候选缓冲区以实现跨层竞争, 从而解决了现有方法中冗余效率低和缺乏横向扩展的问题。

问题表述。先前的研究表明, 草稿模型的输出概率与目标模型的接受概率之间存在显著的正相关。因此 [20], 我们利用草稿模型的预测概率作为置信度的代理, 以估计生成的令牌最终被接受的可能性。设 $\mathcal{V}$ 表示词汇表。给定一个前缀上下文序列

$x_{1:t}$, 草稿模型 M_d 通过 softmax 操作预测下一个标记的概率分布:

$$P(x_{t+1}|x_{1:t}) = \text{Softmax}(\text{Logits}(M_d(x_{1:t}))). \quad (6)$$

我们将 *Token Tree* $\mathcal{T}$ 定义为一组节点, 其中每个节点 u 表示从根节点延伸的一个标记序列 $x_{1:t+d}$ 。考虑到验证过程中固有的顺序依赖性, 即节点 u 的接受需要其所有祖先节点的接受, 路径被目标模型接受的概率近似于沿轨迹所有标记的联合概率。因此, 我们采用累计对数概率 (CLP) 作为度量 $S(u)$ 来量化节点 u 的质量:

$$S(u) = \sum_{k=1}^d \log P(x_{t+k}|x_{1:t+k-1}). \quad (7)$$

我们的目标是构建一棵受严格内存和计算开销限制约束的树 $\mathcal{T}$, 这些限制由 $|\mathcal{T}| \leq K$ 定义, 其中 K 表示令牌预算。具体而言, 该树旨在包含具有最高 $S(u)$ 值的节点集合, 以最大化草稿模型预测分布中高置信度区域的覆盖, 从而提高推测解码的推测成功率。

算法。现有的树构建算法通常执行按层展开, 在深度 d 选择排名前 k 的节点扩展到 $d+1$, 同时立即舍弃其余节点。我们在边云协同推理的背景下识别出这一范式的两个关键限制。首先, 先生成一个大树然后再修剪以找到最优路径的策略会引入过多的计算开销

Algorithm 1: Budget-Aware Global Adaptive Tree Construction

Input: Prefix $x_{1:t}$, Budget K , Max Depth D , Batch Size B , Branch Width W , Admit Rate N_{limit}
Output: Token Tree $\mathcal{T}$

```

1 Initialize Tree  $\mathcal{T} \leftarrow \{x_{1:t}\}$ ,  $Q \leftarrow \{(\emptyset, \text{root})\}$ , Buffer  $\mathcal{B} \leftarrow \emptyset$ ;
2 while  $\text{depth} < D$  and  $|\mathcal{T}| < K$  do
3   // 1. Batch Expansion;
4   Pop batch of parents  $\mathcal{P}_{batch}$  from  $Q$  (size  $\leq B$ );
5   Compute logits for  $\mathcal{P}_{batch}$  via Draft Model;
6   for parent  $u \in \mathcal{P}_{batch}$  do
7     Get top- $W$  children  $\{v_1, \dots, v_W\}$  via LogSoftmax;
8     Calculate scores:  $S(v_i) \leftarrow S(u) + \log P(v_i|u)$ ;
9     Add children to Buffer:  $\mathcal{B} \leftarrow \mathcal{B} \cup \{v_1, \dots, v_W\}$ ;
10  // 2. Global Selection (Lateral Comparison);
11  Sort  $\mathcal{B}$  descending by score  $S(\cdot)$ ;
12   $N_{admit} \leftarrow \min(|\mathcal{B}|, N_{limit}, K - |\mathcal{T}|)$ ;
13   $\mathcal{C}_{best} \leftarrow$  Top  $N_{admit}$  nodes from  $\mathcal{B}$ ;
14  Remove  $\mathcal{C}_{best}$  from  $\mathcal{B}$ ;
15  // 3. Tree Update;
16  for node  $v \in \mathcal{C}_{best}$  do
17    Add  $v$  to  $\mathcal{T}$ ;
18    Push  $v$  to  $Q$  for future expansion;
19  if  $N_{admit} = 0$  then
20    break;
21 return  $\mathcal{T}$ 

```

redundancy. On resource-constrained edge devices, calculating logits for nodes that are ultimately pruned constitutes a significant waste of computational power. Second, confidence decay manifests in two dimensions: depth (where probability decreases as sequence length increases) and breadth (where the probability of a Rank-2 token is lower than that of a Rank-1 token within the same layer). Existing methods tend to prioritize depth, often exclusively extending the optimal branch. However, a high-probability node situated at a shallow layer may possess a higher global likelihood $S(u)$ than a node at a deeper layer where confidence has degraded. Neglecting these shallow nodes results in a tree structure that fails to achieve global optimality, as seen in Fixed-Shape and Layer-wise expansions (Fig. 3(a) and (b)).

To address these drawbacks, we propose implementing lateral expansion via a global buffer pool, achieving the Global Adaptive Expansion shown in Fig. 3(c). Instead of restricting comparisons to within the same hierarchy, we allow candidate nodes from shallow layers to compete directly with those from deeper layers. Consequently, if a deep branch traverses into a low-confidence region, the algorithm dynamically pivots to expand previously buffered high-probability nodes from shallower layers.

The detailed procedure is outlined in Algorithm 1, which

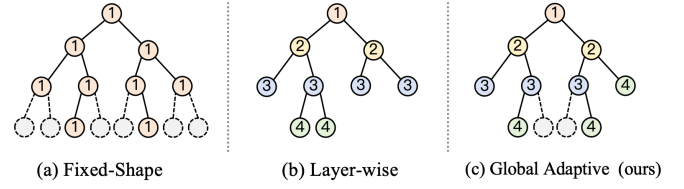


Fig. 3: Comparison of tree expansion strategies. (a) Fixed-Shape expansion first expands aggressively into a large, regular tree and then prunes it, wasting computation. (b) Layer-wise expansion selects only the best node(s) within the current layer at each step. (c) Our global adaptive expansion dynamically chooses the highest-confidence nodes from a global buffer, optimizing the resulting token-tree structure.

orchestrates the generation process through two primary components: a *Global Candidate Buffer* ($\mathcal{B}$) and a *Priority Queue* (Q). The process commences with the initialization of the tree $\mathcal{T}$ containing the input prefix, while Q is seeded with the root node to regulate the expansion sequence. In each iterative cycle, a batch of parent nodes is retrieved from Q to undergo a parallel forward pass via the draft model. For each parent, the probabilities of the top- W child tokens are computed. Crucially, rather than being immediately appended to the tree, these potential nodes are deposited into the Global Candidate Buffer $\mathcal{B}$ alongside their cumulative scores $S(u)$. Subsequently, a global selection mechanism is employed, wherein the entire buffer—comprising both newly generated candidates and unselected options from previous layers—is sorted by score. The top- N_{limit} nodes are admitted into $\mathcal{T}$ and moved from $\mathcal{B}$ to Q for subsequent expansion, while unselected nodes remain in $\mathcal{B}$. This enables lateral exploration by preserving high-probability shallow candidates when deeper paths become less promising.

D. Asynchronous Pipelining

While the pure index protocol minimizes transmission latency, the computational latency inherent in target model verification is unavoidable, resulting in an idle window (or “bubble”) where the edge device awaits the response (Fig. 4). To bridge this gap and fully exploit edge computational resources, we propose a *Batched Pre-Drafting* mechanism. This approach enables the edge to speculatively generate potential drafts for the subsequent round while concurrently awaiting verification. Upon receiving the cloud’s response, if a finalized path aligns with a branch that was speculatively extended during the waiting period, the system can bypass the drafting phase for the next cycle and immediately transmit the pre-computed tree. The overall procedure is summarized in Algorithm 2.

Candidate Tree Initialization. Rather than simply deepening the current token tree $\mathcal{T}_{curr}$, we opt to pre-construct independent *Candidate Trees* rooted at the high-confidence leaf nodes of $\mathcal{T}_{curr}$. This strategy ensures that the concurrently generated candidate trees structurally align with the drafts that would otherwise be generated *post-verification*, effectively

Algorithm 1: Budget-Aware Global Adaptive Tree Construction

Input: Prefix $x_{1:t}$, Budget K , Max Depth D , Batch Size B , Branch Width W , Admit Rate N_{limit}
Output: Token Tree $\mathcal{T}$

```

1 Initialize Tree  $\mathcal{T} \leftarrow \{x_{1:t}\}$ ,  $Q \leftarrow \{(0, \text{root})\}$ , Buffer  $\mathcal{B} \leftarrow \emptyset$ ;
2 while depth <  $D$  and  $|\mathcal{T}| < K$  do
3   // 1. Batch Expansion;
4   Pop batch of parents  $\mathcal{P}_{batch}$  from  $Q$  (size  $\leq B$ );
5   Compute logits for  $\mathcal{P}_{batch}$  via Draft Model;
6   for parent  $u \in \mathcal{P}_{batch}$  do
7     Get top- $W$  children  $\{v_1, \dots, v_W\}$  via LogSoftmax;
8     Calculate scores:  $S(v_i) \leftarrow S(u) + \log P(v_i|u)$ ;
9     Add children to Buffer:  $\mathcal{B} \leftarrow \mathcal{B} \cup \{v_1, \dots, v_W\}$ ;
10  // 2. Global Selection (Lateral Comparison);
11  Sort  $\mathcal{B}$  descending by score  $S(\cdot)$ ;
12   $N_{admit} \leftarrow \min(|\mathcal{B}|, N_{limit}, K - |\mathcal{T}|)$ ;
13   $\mathcal{C}_{best} \leftarrow$  Top  $N_{admit}$  nodes from  $\mathcal{B}$ ;
14  Remove  $\mathcal{C}_{best}$  from  $\mathcal{B}$ ;
15  // 3. Tree Update;
16  for node  $v \in \mathcal{C}_{best}$  do
17    Add  $v$  to  $\mathcal{T}$ ;
18    Push  $v$  to  $Q$  for future expansion;
19  if  $N_{admit} = 0$  then
20    break;
21 return  $\mathcal{T}$ 
  
```

冗余。在资源受限的边缘设备上，为最终将被剪枝的节点计算 logits 会造成大量计算资源的浪费。其次，置信度衰减表现为两个维度：深度（随着序列长度增加，概率下降）和宽度（在同一层中，Rank-2 令牌的概率低于 Rank-1 令牌）。现有方法倾向于优先考虑深度，通常仅扩展最优分支。然而，位于浅层的高概率节点可能具有比位于深层、置信度已经降低的节点更高的全局可能性 $S(u)$ 。忽略这些浅层节点会导致树结构无法实现全局最优性，如固定形状和按层扩展中所示（图 3(a) 和 (b)）。

为了解决这些缺点，我们建议通过全局缓冲池实现横向扩展，从而实现图 3(c) 所示的全局自适应扩展。我们不再将比较限制在同一层级内，而是允许浅层的候选节点与深层的节点直接竞争。因此，如果一个深层分支进入低置信度区域，算法会动态地转向展开以前缓冲的浅层高概率节点。

详细的过程在算法1中有说明，它

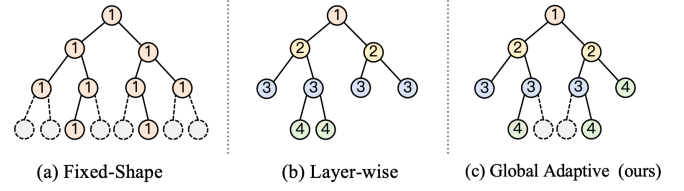


图3：树扩展策略比较。(a) 固定形状扩展首先积极扩展成一个大且规则的树，然后再进行剪枝，浪费计算资源。(b) 分层扩展在每一步只选择当前层中最优的节点。(c) 我们的全局自适应扩展从全局缓冲区动态选择最高置信度的节点，优化生成的令牌树结构。

通过两个主要组件协调生成过程：一个 *Global Candidate Buffer* ($\mathcal{B}$) 和一个 *Priority Queue* (Q)。该过程从初始化包含输入前缀的树 $\mathcal{T}$ 开始，同时用根节点为 Q 提供初始值以调节扩展顺序。在每个迭代循环中，从 Q 获取一批父节点，以通过草稿模型进行并行前向计算。对于每个父节点，计算前 W 个子令牌的概率。关键在于，这些潜在节点不会立即附加到树中，而是与它们的累积分数 $S(u)$ 一起存入全局候选缓冲区 $\mathcal{B}$ 。随后，采用全局选择机制，将整个缓冲区——包括新生成的候选节点和前一层未选中的选项——按分数排序。前 N_{limit} 个节点被加入 $\mathcal{T}$ 并从 $\mathcal{B}$ 移动到 Q 以供后续扩展，而未被选中的节点则保留在 $\mathcal{B}$ 中。这实现了横向扩展

D. Asynchronous Pipelining

虽然纯索引协议可以最小化传输延迟，但目标模型验证中固有的计算延迟是不可避免的，从而产生一个空闲窗口（或“气泡”），边缘设备在此期间等待响应（图 4）。为了弥补这一差距并充分利用边缘计算资源，我们提出了一种 *Batched Pre-Drafting* 机制。该方法使边缘设备能够在等待验证的同时，推测性地生成下一轮的潜在草稿。在收到云端响应后，如果最终确定的路径与等待期间推测性扩展的分支一致，系统可以跳过下一轮的起草阶段，直接传输预计算的树。整体过程总结见算法 2。

候选树初始化。我们不只是简单地加深当前的标记树 $\mathcal{T}_{curr}$ ，而是选择在 $\mathcal{T}_{curr}$ 的高置信叶节点上预先构建独立的 *Candidate Trees*。这一策略确保同时生成的候选树在结构上与原本将生成的草稿 *post-verification* 对齐，有效地

Algorithm 2: Batched Pre-Drafting with Asynchronous Verification

Input: Current tree $\mathcal{T}_{curr}$, draft model $\mathcal{M}_d$, cloud model $\mathcal{M}_c$, top- k k

- 1 send $\mathcal{T}_{curr}$ to Cloud;
 - 2 **async (Cloud):** $(\mathcal{R}, l_{acc}) \leftarrow \text{Verify}(\mathcal{T}_{curr}, \mathcal{M}_c)$;
 - 3 **async (Edge):** $\mathcal{T}' \leftarrow \text{Expand}(\mathcal{T}_{curr}, \mathcal{M}_d, 1)$;
 - 4 $\mathcal{L} \leftarrow \text{TopKLeaves}(\mathcal{T}', k)$;
 - 5 $\mathbb{S} \leftarrow \{\text{InitTree}(l) : l \in \mathcal{L}\}$;
 - 6 **repeat:** $\mathcal{B} \leftarrow \text{Frontier}(\mathbb{S})$;
 - 7 $\mathbb{S} \leftarrow \text{BatchExpand}(\mathcal{B}, \mathcal{M}_d)$;
 - 8 **until** $\mathcal{R}$ received;
 - 9 $l_{acc} \leftarrow \text{AcceptedLeaf}(\mathcal{R})$;
 - 10 $\mathcal{T}_{next} \leftarrow \begin{cases} \text{Retrieve}(\mathbb{S}, l_{acc}), & l_{acc} \in \mathcal{L} \\ \text{StandardDraft}(l_{acc}, \mathcal{M}_d), & \text{otherwise} \end{cases}$;
 - 11 send $\mathcal{T}_{next}$ to Cloud if $l_{acc} \in \mathcal{L}$;
-

anticipating the cloud’s acceptance decision. This parallelization aims to synchronize the drafting of the next round with the verification of the current round, thereby eliminating the bubble caused by the cloud waiting for the edge’s drafting phase. Let $\mathcal{L} = \{l_1, l_2, \dots, l_N\}$ denote the set of leaf nodes in $\mathcal{T}_{curr}$. After transmitting $\mathcal{T}_{curr}$ to the cloud, the edge device performs a single-step expansion on $\mathcal{T}_{curr}$ —accounting for the potential bonus token returned by the cloud—to obtain $\mathcal{T}'_{curr}$. Subsequently, a subset of high-confidence leaf nodes $\mathcal{L}_{top} \subset \mathcal{T}'_{curr}$ is selected as potential starting points for the next inference step. For each selected leaf $l_i \in \mathcal{L}_{top}$, a corresponding candidate tree $\mathcal{T}'_i$ is initialized, where the root of $\mathcal{T}'_i$ logically connects to l_i , representing a hypothetical path continuation contingent upon the acceptance of l_i .

Batched Expansion and Activation. To efficiently construct multiple candidate trees simultaneously on resource-constrained edge devices [21], we leverage the *Batch Processing* capability of the draft model. During the expansion at each depth d , candidate nodes from all candidate trees $\{\mathcal{T}'_1, \dots, \mathcal{T}'_k\}$ are aggregated into a single inference batch. This batched approach amortizes the overhead associated with kernel launches and memory I/O [22], ensuring that the total latency for concurrently building k candidate trees remains significantly lower than that of sequential execution. Upon receipt of the verification response $\mathcal{R}$ from the cloud, the edge device immediately performs a matching operation. If the accepted path terminates at a leaf node $l^* \in \mathcal{L}_{top}$, the corresponding candidate tree $\mathcal{T}'_{l^*}$ is instantly “activated.” This pre-computed tree is immediately serialized and transmitted to the cloud, effectively eliminating the drafting latency for the subsequent round (i.e., $T_{draft} \rightarrow 0$). If the accepted path matches no pre-specified leaf, the candidate trees are discarded and the edge resumes standard drafting from the corrected tokens. By overlapping candidate drafting with cloud verification, the framework improves throughput and keeps edge computation active during communication and verification intervals.

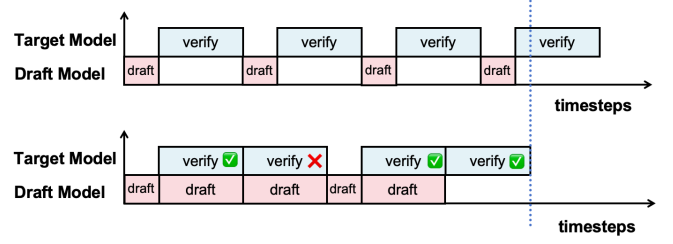


Fig. 4: Timeline comparison between standard sequential inference (top) and our asynchronous pipelining (bottom). In the standard approach, the draft model idles while the target model verifies. Our method leverages this interval to perform pre-drafting, effectively masking the verification latency.

IV. EVALUATION

Experimental Setup. We evaluate the proposed framework in a heterogeneous edge-cloud collaborative computing environment. The cloud server is equipped with eight NVIDIA A100 GPUs (40GB) hosting the large-scale target model (M_p), representing a high-performance centralized infrastructure, while the edge node is a consumer-grade workstation with two NVIDIA RTX 4090 GPUs (24GB) running the lightweight draft model (M_q), simulating a typical edge device. The system is implemented in PyTorch [23], and all interactions between the edge and cloud nodes are conducted via *gRPC* to minimize communication overhead and ensure reliable transmission. All experiments are performed under a wide-area network (WAN) setting with a maximum bandwidth of 200 Mbps to emulate real-world deployment conditions. To assess the generalization and effectiveness of the proposed method, we conduct extensive evaluations across multiple model pairs and standard benchmarks.

Evaluation Metrics. The key evaluation metric is the *token throughput* of the system, formally defined as:

$$\text{Throughput} = \sum_{i=1}^{\text{Batch Size}} \text{Token-Length}_i / \text{Inference-Latency} \quad (8)$$

It is calculated by dividing the total number of tokens generated (corresponding to Batch Size parallel requests each producing Token Length tokens) by the total latency consumed for completing this batch of inference tasks. Specifically, the batch size is adaptively set to occupy a fixed GPU memory footprint on the cloud side, enabling fair comparison across different settings in edge-cloud collaborative and cloud-only scenarios.

Beyond throughput, we also quantify the serving *costs* based on prevailing market rental prices: \$4.05 per hour for an A100 40G GPU and \$0.35 per hour for an RTX 4090 GPU [24], [25]. In the ablation study, we verify the effectiveness of our tree-based drafting algorithm using *accept rate*, which is the proportion of the final generated tokens that are produced by the draft model and accepted by the target LLM.

Baselines. To comprehensively evaluate the effectiveness of our proposed framework, we compare it against *two cloud-only* inference methods as well as *four state-of-the-art collaborative*

Algorithm 2: Batched Pre-Drafting with Asynchronous Verification

Input: Current tree $\mathcal{T}_{curr}$, draft model $\mathcal{M}_d$, cloud model $\mathcal{M}_c$, top- k k

- 1 send $\mathcal{T}_{curr}$ to Cloud;
 - 2 **async (Cloud):** $(\mathcal{R}, l_{acc}) \leftarrow \text{Verify}(\mathcal{T}_{curr}, \mathcal{M}_c)$;
 - 3 **async (Edge):** $\mathcal{T}' \leftarrow \text{Expand}(\mathcal{T}_{curr}, \mathcal{M}_d, 1)$;
 - 4 $\mathcal{L} \leftarrow \text{TopKLeaves}(\mathcal{T}', k)$;
 - 5 $\mathbb{S} \leftarrow \{\text{InitTree}(l) : l \in \mathcal{L}\}$;
 - 6 **repeat:** $\mathcal{B} \leftarrow \text{Frontier}(\mathbb{S})$;
 - 7 $\mathbb{S} \leftarrow \text{BatchExpand}(\mathcal{B}, \mathcal{M}_d)$;
 - 8 **until** $\mathcal{R}$ received;
 - 9 $l_{acc} \leftarrow \text{AcceptedLeaf}(\mathcal{R})$;
 - 10 $\mathcal{T}_{next} \leftarrow \begin{cases} \text{Retrieve}(\mathbb{S}, l_{acc}), & l_{acc} \in \mathcal{L} \\ \text{StandardDraft}(l_{acc}, \mathcal{M}_d), & \text{otherwise} \end{cases}$;
 - 11 send $\mathcal{T}_{next}$ to Cloud if $l_{acc} \in \mathcal{L}$;
-

预测云端的接受决策。此并行化旨在将下一轮的草拟与当前轮的验证同步，从而消除云端等待边缘设备草拟阶段所产生的空白。令 $\mathcal{L} = \{l_1, l_2, \dots, l_N\}$ 表示 $\mathcal{T}_{curr}$ 中的叶节点集合。在将 $\mathcal{T}_{curr}$ 传输到云端后，边缘设备在 $\mathcal{T}_{curr}$ 上执行单步扩展——考虑到云端可能返回的奖励 token——以获得 $\mathcal{T}'_{curr}$ 。随后，从中选择一部分高置信度叶节点 $\mathcal{L}_{top} \subset \mathcal{T}'_{curr}$ 作为下一步推理的潜在起点。对于每个选定的叶节点 $l_i \in \mathcal{L}_{top}$ ，初始化对应的候选树 $\mathcal{T}'_i$ ，其中 $\mathcal{T}'_i$ 的根逻辑上连接到 l_i ，表示在 l_i 被接受的情况下假设的路径延续。

批处理扩展与激活。为了在资源受限的边缘设备上高效地同时构建多个候选树[21]，我们利用了草稿模型的 *Batch Processing* 能力。在每个深度 d 的扩展过程中，来自所有候选树的候选节点 $\{\mathcal{T}'_1, \dots, \mathcal{T}'_k\}$ 会被聚合到单个推理批次中。这种批处理方法分摊了与内核启动和内存 I/O 相关的开销[22]，确保同时构建 k 候选树的总延迟显著低于顺序执行。当收到来自云端的验证响应 $\mathcal{R}$ 时，边缘设备会立即执行匹配操作。如果被接受的路径在叶节点 $l^* \in \mathcal{L}_{top}$ 处终止，则相应的候选树 $\mathcal{T}'_{l^*}$ 会立即“激活”。这个预计算的树会被立即序列化并传输到云端，有效地消除了下一轮（即 $T_{draft} \rightarrow 0$ ）的草稿延迟。如果接受的路径

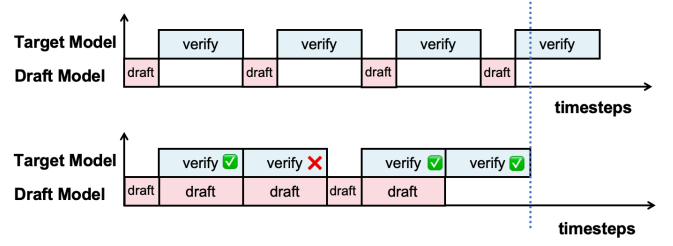


图4: 标准顺序推理（上）与我们的异步流水线（下）的时间线比较。在标准方法中，草稿模型在目标模型进行验证时处于空闲状态。我们的方法利用这一间隔进行预草稿，有效地掩盖了验证延迟。

四、评估

实验设置。我们在异构边缘-云协同计算环境中评估所提出的框架。云服务器配备八块 NVIDIA A100 GPU (40G B)，承载大规模目标模型 (M_p)，代表高性能的集中式基础设施，而边缘节点为一台消费级工作站，配备两块 NVIDIA RTX 4090 GPU (24GB)，运行轻量级草案模型 (M_q)，模拟典型的边缘设备。系统使用 PyTorch [23] 实现，边缘节点与云节点之间的所有交互均通过 *gRPC* 进行，以最小化通信开销并确保可靠传输。所有实验均在广域网 (WAN) 设置下进行，最大带宽为 200 Mbps，以模拟真实部署环境。为了评估所提方法的泛化能力和有效性，我们在多个模型对和标准基准上进行了广泛评估。

评估指标。关键评估指标是系统的 *token throughput*，正式定义如下：

$$\text{Throughput} = \sum_{i=1}^{\text{Batch Size}} \text{Token-Length}_i / \text{Inference-Latency} \quad (8)$$

它通过将生成的令牌总数（对应于每个并行请求生成令牌长度的批量大小）除以完成这一批次推理任务所消耗的总延迟来计算。具体来说，批量大小会自适应设置，以占用云端固定的 GPU 内存，从而在边云协同和仅云场景下实现不同设置之间的公平比较。

除了吞吐量，我们还根据现行市场租赁价格量化服务 *costs*：A100 40G GPU 为每小时 4.05 美元，RTX 4090 GPU 为每小时 0.35 美元 [24], [25]。在消融研究中，我们使用 *accept rate* 验证了基于树的草稿算法的有效性，*accept rate* 是最终生成的 token 中由草稿模型生成并被目标大语言模型接受的比例。

基线。为了全面评估我们所提出框架的有效性，我们将其与 *two cloud-only* 推理方法以及 *four state-of-the-art collaborative* 进行比较

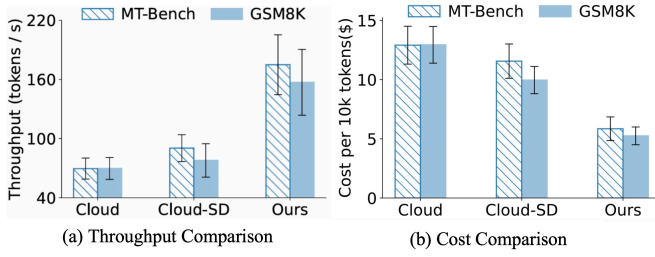


Fig. 5: Comparison of cloud-only LLM inference and our method on the dataset of MT-Bench and GSM8K.

TABLE I: Comparison in terms of cloud memory and latency.

Method	Cloud Model Memory	Latency	Throughput
Cloud	121 GB	16.4 s	69.5 tokens/s
Cloud-SD	146 GB	5.36 s	90.3 tokens/s
Ours	121 GB	6.20 s	175 tokens/s

methods. The two representative cloud-only methods are as follows. (1) *Cloud-Only Autoregressive Generation*, where the target model executes entirely on the server, serving as the standard baseline for inference latency. (2) *Cloud-Only Speculative Decoding (Cloud-SD)* [5], where both the draft model and target LLMs are deployed on the cloud to perform speculative decoding. And we configure the batch size for Cloud-SD to 5 to ensure stable execution.

Then we choose four state-of-the-art collaborative methods. (1) *Model Split* [26], a traditional offloading strategy that partitions the model layers between the edge device and the cloud. (2) *Distributed Speculative Decoding (DSD)* [13], which applies standard speculative decoding in a distributed setting, necessitating the transmission of full probability distributions. (3) *DSSD* [9], which optimizes communication latency of the collaborative speculative decoding via splitting the verification phases. (4) *TKSLT* [10], which mitigates communication bottlenecks in collaborative speculative decoding by transmitting only the top-k sparse logits.

Models and Datasets. To demonstrate the scalability of our framework across different parameter scales, we conduct evaluations using the Vicuna family of models [27], which are fine-tuned from LLaMA models [2]. We establish two distinct draft-target pairs: Vicuna-33B paired with Vicuna-7B, and Vicuna-7B paired with Vicuna-68M. In terms of hardware allocation, the 33B model is distributed across 8 GPUs to accommodate its massive memory footprint, whereas the 7B model and 68M model are deployed on a single GPU. We assess these models on two widely used benchmarks to cover diverse capabilities, specifically utilizing MT-Bench [28] for multi-turn conversation and instruction following, and GSM8K [29] for mathematical reasoning tasks. For GSM8K, we use only the first 100 test examples for evaluation. Unless specifically noted, a uniform temperature setting of 1.0 is applied across all experimental configurations, and the maximum generation length is set to 128 tokens.

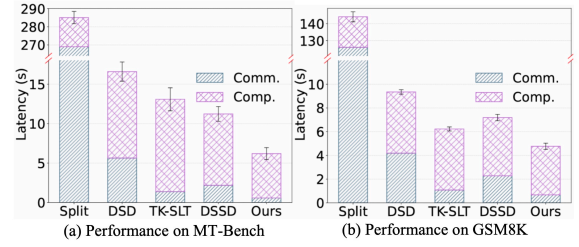


Fig. 6: Communication and computation latency of different collaborative methods on two datasets using 33B-7B models.

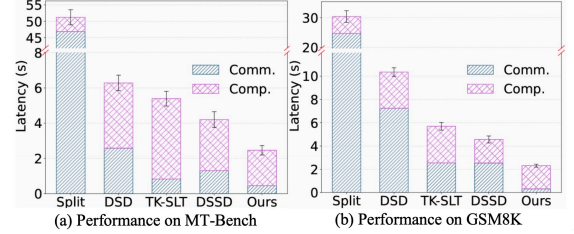


Fig. 7: Latency performance on 7B-68M models.

A. End-to-End Performance and Cost-Efficiency

We first evaluate the end-to-end performance of our proposed framework against cloud-only baselines to demonstrate the core advantages freeing up cloud memory to maximize throughput. Results are shown in Fig. 5 and Table I.

As detailed in Table I, co-deploying both the draft 7B model and target 33B model in the cloud (Cloud-SD) consumes 146 GB of cloud memory. By offloading the draft model to the edge device, our method reduces the cloud memory footprint back to 121 GB, matching the memory usage of standard autoregressive generation (Cloud). Crucially, this saved memory allows the cloud server to accommodate a larger batch size, which translates into a massive throughput advantage: *our approach achieves a higher throughput of 175 tokens/s, which is 2.52 $\times$ of cloud-only autoregression and 1.94 $\times$ of Cloud-SD.* It verifies that, despite the inherent cloud-edge communication latency that makes our method slightly slower compared to Cloud-SD (6.20 s vs. 5.36 s), the significant memory savings translate to a remarkable throughput benefit.

Fig. 5 further illustrates this advantage across different datasets. Compared to Cloud-SD, our edge-cloud collaborative method *consistently achieves about 2 $\times$ throughput gains* on both MT-Bench and GSM8K datasets. Meanwhile, since the draft model computations are offloaded to cost-effective edge devices, *our method separately reduces the token generation cost by about 47.3% and 31.0% on each dataset.* This indicates that our method is promising as an economical solution for fast and large-scale LLM serving.

B. Latency Breakdown and Collaborative Efficiency

To thoroughly compare with existing edge-cloud collaborative methods, we break down the total inference time into communication and computation latency. Comparison results

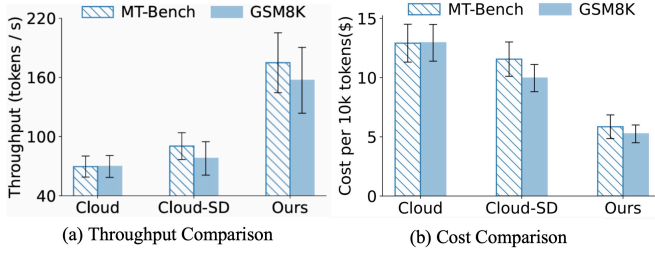


图 5: 在 MT-Bench 和 GSM8K 数据集上, 云端仅 LLM 推理与我们方法的比较。

表 I: 在云存储和延迟方面的比较。

Method	Cloud Model Memory	Latency	Throughput
Cloud	121 GB	16.4 s	69.5 tokens/s
Cloud-SD	146 GB	5.36 s	90.3 tokens/s
Ours	121 GB	6.20 s	175 tokens/s

方法。两个典型的纯云方法如下: (1)

Cloud-Only Autoregressive Generation, 其中目标模型完全在服务器上执行, 作为推理延迟的标准基准。(2)

Cloud-Only Speculative Decoding (Cloud-SD) [5], 其中草稿模型和目标大语言模型都部署在云端以执行预测解码。我们将 Cloud-SD 的批量大小配置为 5 以确保稳定执行。

然后我们选择了四种最先进的协作方法。(1)

Model Split [26], 一种传统的卸载策略, 将模型层在边缘设备和云之间进行划分。(2)

Distributed Speculative Decoding (DSD) [13], 在分布式环境中应用标准投机解码, 需要传输完整的概率分布。(3)

DSSD [9], 通过拆分验证阶段优化协作投机解码的通信延迟。(4) *TKSLT* [10], 通过仅传输前k个稀疏logits来缓解协作投机解码中的通信瓶颈。

模型和数据集。为了展示我们框架在不同参数规模下的可扩展性, 我们使用 Vicuna 系列模型 [27] 进行评估, 这些模型是在 LLaMA 模型 [2] 的基础上微调的。我们建立了两个不同的草稿-目标配对: Vicuna-33B 配对 Vicuna-7B, 以及 Vicuna-7B 配对 Vicuna-68M。在硬件分配方面, 33 B 模型分布在 8 块 GPU 上以适应其巨大的内存占用, 而 7 B 模型和 68M 模型则部署在单块 GPU 上。我们在两个广泛使用的基准测试上评估这些模型, 以覆盖不同的能力, 具体使用 MT-Bench [28] 进行多轮对话和指令遵循评测, 以及 GSM8K [29] 进行数学推理任务评测。对于 GSM8K, 我们仅使用前 100 个测试样本进行评估。除非另有说明, 所有实验配置均采用统一的温度设置为 1.0, 并将最大生成长度设置为 128 个标记。

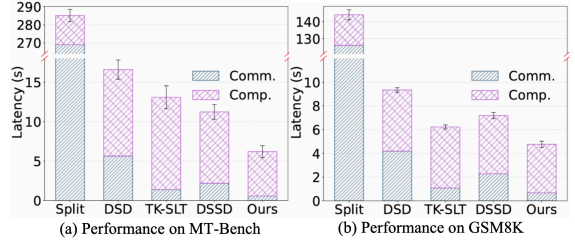


图6: 在使用33B-7B模型的两个数据集上, 不同协作方法的通信和计算延迟。

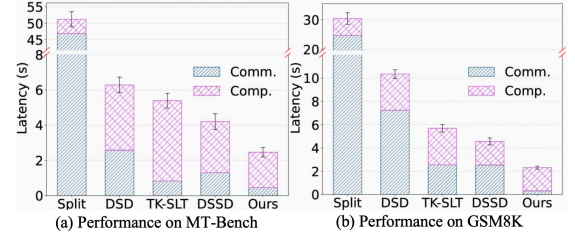


图7: 7B-68M模型的延迟性能。

A. End-to-End Performance and Cost-Efficiency

我们首先评估所提出框架的端到端性能, 并与仅云端的基线进行比较, 以展示释放云端内存以最大化吞吐量的核心优势。结果如图 5 和表 I 所示。

如表 I 所示, 在云端同时部署草稿 7B 模型和目标 33B 模型 (Cloud-SD) 需要 146 GB 的云内存。通过将草稿模型卸载到边缘设备, 我们的方法将云内存占用减少回 121 GB, 与标准自回归生成 (Cloud) 的内存使用量相当。关键在于, 这部分节省的内存允许云服务器容纳更大的批量大小, 这转化为巨大的吞吐量优势: *our approach achieves a higher throughput of 175 tokens/s, which is $2.52\times$ of cloud-only autoregression and $1.94\times$ of Cloud-SD*。这验证了, 尽管固有的云-边缘通信延迟使我们的方法相比 Cloud-SD 略慢 (6.20 秒 vs. 5.36 秒), 但显著的内存节省带来了卓越的吞吐量收益。

图5进一步说明了这一优势在不同数据集上的表现。与 Cloud-SD相比, 我们的边缘-云协作方法 *consistently achieves about $2\times$ throughput gains* 在 MT-Bench 和 GSM8K 数据集上均表现良好。同时, 由于草稿模型的计算被卸载到成本效益高的边缘设备上, *our method separately reduces the token generation cost by about 47.3% and 31.0% on each dataset*。这表明我们的方法作为一种经济的快速大规模 LLM 服务解决方案具有很大潜力。

B. Latency Breakdown and Collaborative Efficiency

为了与现有的边缘-云协作方法进行全面比较, 我们将总推理时间分解为通信延迟和计算延迟。比较结果

TABLE II: Ablation Study.

Method	Throughput (token/s)	Cost _{100k} (\$)
w/o Tree	143.70	6.51
w/o Parallel	148.56	6.30
Full	174.94	5.35

against Split, DSD, TK-SLT, and DSSD are shown in Fig. 6 and Fig. 7, using 33B-7B models and 7B-68M models.

Communication Latency. Due to the large intermediate features of LLMs, the Split approach incurs a transmission volume that is dozens of times higher than that of other methods, which severely impacts the corresponding latency. The other collaborative speculative decoding methods all suffer from the substantial communication overhead of logits transmission. On the contrary, *our token-only communication protocol consistently reduces the communication latency by over 38.4% across diverse datasets using LLM models of varying sizes.* As a result, our method visibly outperforms all state-of-the-art methods in overall latency.

Computation Latency. Beyond communication savings, our method also demonstrates superior computation latency. On the MT-Bench dataset, where the task is more challenging with higher computational complexity, this superiority becomes more pronounced. Specifically, *compared to the state-of-the-art methods, our computation latency of 33B-7B model and 7B-68M model are reduced by 37.9% and 30.5%, respectively.* This is attributed to two key designs. First, our budget-aware tree drafting strategy significantly improves the acceptance rate of candidate tokens, thereby reducing the total number of required interaction rounds between the edge and the cloud. Second, our asynchronous pipelining mechanism effectively overlaps the edge draft model’s computation time with the cloud’s verification time, which masking this computational overhead by minimizing the idle bubbles.

Therefore, our method consistently achieves the lowest overall latency across diverse task complexities (MT-Bench and GSM8K) and varying model scales (33B-7B and 7B-68M), and *reduces the overall inference latency by up to 49.3%.* This verifies its strong generalization capabilities in real-world edge-cloud environments.

C. Ablation Study

To comprehensively understand the contributions of each components and the performance under various hyperparameters, we conduct detailed ablation studies and evaluations.

Effectiveness of Core Components. We first ablate the two fundamental components of our framework: the tree-based draft structure and the asynchronous pipelining mechanism. As shown in Table II, the full Ygg framework achieves the highest throughput of 174.94 tokens/s and the lowest inference cost. The configuration without tree structures (*w/o Tree*) exhibits a severe performance drop, achieving a throughput of only 143.70 tokens/s. This substantial degradation indicates that, without a tree-based structure, the system degenerates into a fragile single-sequence drafting paradigm. Such a linear

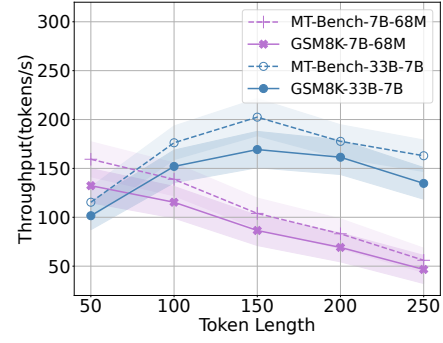


Fig. 8: Performance with different sequence length.

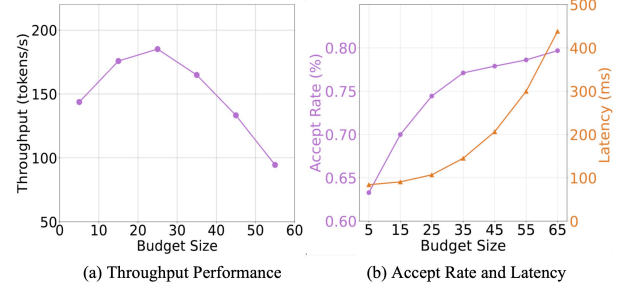


Fig. 9: Impact of budget size choice. As the budget increases, the acceptance rate rises, but the transmission latency also increases, so the throughput first increases and then decreases.

approach fails to exploit the cloud’s parallel verification capability, resulting in lower token acceptance rates and increased communication rounds. Furthermore, the configuration without parallel execution (*w/o Parallel*) drops to 148.56 tokens/s. This performance gap reveals the presence of an idle computational “bubble” during the cloud verification phase under standard sequential execution. By leveraging our algorithm to pre-draft candidate trees, the full framework effectively overlaps drafting with verification, masking the latency and maximizing throughput.

Impact of Sequence Length. As shown in Eq. 8, the throughput is affected by the length of generated tokens. We separately set the maximum token length to different values and plot the throughput in Fig. 8. It’s observed that for large LLM models, *i.e.*, 33B-7B, the throughput performance remains stable above 150 tokens/s across different sequence lengths. While for 7B-68M model pair, we observe a continuous downward trend in throughput as the sequence length increases. The kernel reason is that the release of the 68M draft model saves relatively little GPU memory, which hardly achieves sufficient batch size gains for long sequences. *This implies that our approach yields more stable acceleration for larger LLM models, which releases substantial GPU memory.* As models continue to grow in complexity, it is expected to play a more important role in the future.

Impact of Budget Size. Fig. 9 illustrates the trade-off introduced by the tree budget size. As the budget increases, the edge device is permitted to transmit more candidate tokens. This improves the acceptance rate as in Fig. 9 (b). However, an

表 II: 消融研究。

Method	Throughput (token/s)	Cost _{100k} (\$)
w/o Tree	143.70	6.51
w/o Parallel	148.56	6.30
Full	174.94	5.35

针对 Split、DSD、TK-SLT 和 DSSD 的情况如图 6 和图 7 所示, 使用了 33B-7B 模型和 7B-68M 模型。

通信延迟。由于大语言模型 (LLM) 中间特征较大, 分割 (Split) 方法产生的传输量比其他方法高出数十倍, 这严重影响了相应的延迟。其他协同推测解码方法都受到 logits 传输带来的大量通信开销的影响。相反,

our token-only communication

protocol consistently reduces the communication latency by over 38.4% across diverse datasets using LLM models of varying sizes. 因此, 我们的方法在整体延迟上明显优于所有最先进的方法。

计算延迟。除了通信节约之外, 我们的方法还展示了优越的计算延迟。在 MT-Bench 数据集上, 由于任务更具挑战性且计算复杂度更高, 这种优势变得更加明显。具体来说, *compared to the state-of-the-art methods, our computation latency of 33B-7B model and 7B-68M model are reduced by 37.9% and 30.5%, respectively.* 这归因于两个关键设计。首先, 我们的预算感知树草稿策略显著提高了候选令牌的接受率, 从而减少了边缘与云之间所需的总交互轮次。其次, 我们的异步流水线机制有效地将边缘草稿模型的计算时间与云端的验证时间重叠, 从而通过最小化空闲间隙掩盖了这种计算开销。

因此, 我们的方法在不同任务复杂度 (MT-Bench 和 GS M8K) 和不同模型规模 (33B-7B 和 7B-68M) 下始终实现了最低的整体延迟, 并且 *reduces the overall inference latency by up to 49.3%.* 这验证了其在真实边缘云环境中的强大泛化能力。

C. Ablation Study

为了全面理解每个组件的贡献以及在各种超参数下的性能, 我们进行了详细的消融研究和评估。

核心组件的有效性。我们首先消融了框架的两个基本组件: 基于树的草稿结构和异步流水线机制。如表 II 所示, 完整的 Ygg 框架实现了最高的吞吐量 174.94 令牌/秒和最低的推理成本。没有树结构的配置 (w/o Tree) 表现出严重的性能下降, 仅实现 143.70 令牌/秒的吞吐量。这种显著的退化表明, 没有基于树的结构, 系统会退化为脆弱的单序列草稿范式。这种线性

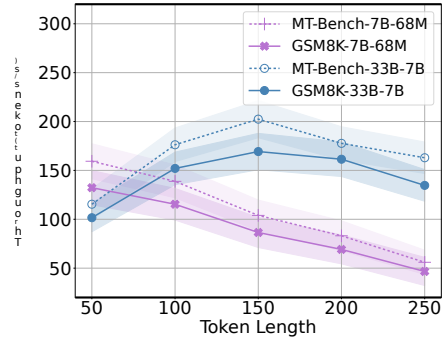


图 8: 不同序列长度下的性能。

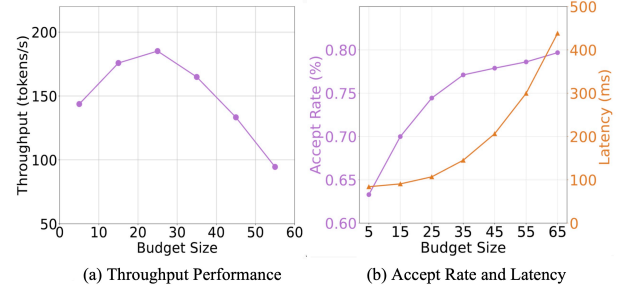


图 9: 预算规模选择的影响。随着预算的增加, 接受率上升, 但传输延迟也增加, 因此吞吐量先增加后减少。

该方法未能利用云的并行验证能力, 导致令牌接受率较低且通信轮次增加。此外, 在没有并行执行的配置下 (w/o Parallel) 速度降至 148.56 个令牌/秒。这个性能差距显示在标准顺序执行下, 云验证阶段存在计算空闲的“气泡”。通过利用我们的算法预先起草候选树, 完整框架可以有效地将起草与验证重叠, 从而掩盖延迟并最大化吞吐量。

序列长度的影响。如公式 8 所示, 吞吐量受到生成标记长度的影响。我们分别将最大标记长度设置为不同的值, 并在图 8 中绘制了吞吐量。观察到对于大型 LLM 模型, 如 i.e.、33B-7B, 在不同序列长度下吞吐性能保持在 150 tokens/s 以上的稳定水平。而对于 7B-68M 模型对, 则随着序列长度的增加, 吞吐量呈持续下降趋势。其根本原因在于 68M 草稿模型的释放节省的 GPU 内存相对较少, 这几乎无法为长序列获得足够的批量大小提升。This implies that our approach yields more stable acceleration for larger LLM models, which releases substantial GPU memory. 随着模型复杂度的不断增加, 预计其在未来将发挥更重要的作用。

预算规模的影响。图 9 展示了树预算规模引入的权衡。随着预算的增加, 边缘设备可以传输更多的候选令牌。这提高了接受率, 如图 9 (b) 所示。然而,

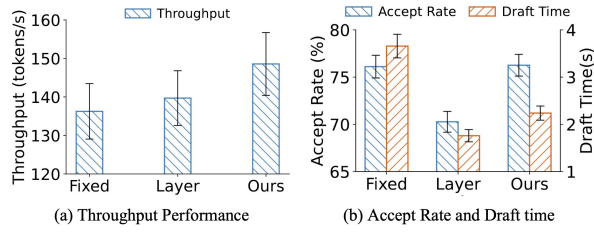


Fig. 10: Impact of tree construction algorithm.

expanded budget simultaneously increases the computational and transmission latency of a single interactive round (drafting, verification, and communication). Consequently, as shown in Fig. 9 (a), *the throughput exhibits a concave trend: it initially increases as the benefits of a higher acceptance rate dominate, but eventually decreases when the per-round latency overhead outweighs the reduction in interaction rounds.*

Impact of Tree Construction Algorithm. Fig. 10 compares our Budget-Aware Global Adaptive Tree Construction against standard Fixed and Layer-wise expansion algorithms. The *Fixed* strategy aggressively expands a large, regular tree before pruning; while this exhaustive approach yields the highest token acceptance rate, its throughput is bottlenecked by the massive redundant draft time overhead on the edge device. Conversely, the *Layer-wise* approach yields a significantly lower acceptance rate because it restricts node selection strictly within current layers, frequently discarding globally optimal nodes located at shallower depths. Our proposed global adaptive algorithm effectively resolves this dilemma. By dynamically selecting high-confidence nodes from a global buffer, it maintains a highly competitive acceptance rate with acceptable level of computational redundancy. As a result, it achieves a better trade-off and delivers the highest overall throughput among the three strategies.

V. RELATED WORK

Efficient On-Device Inference. To mitigate the heavy deployment overhead of LLMs on centralized clouds, recent research has pivoted towards enabling local inference. However, existing workarounds fail to address this issue without significant compromises. Specifically, weight quantization methods [30], [31] successfully compress the memory footprint but typically incur an irreversible degradation in generation quality. Alternatively, substituting them with edge-native lightweight models facilitates local execution [32], [33], yet their reasoning capabilities fall drastically short of their large-scale counterparts. Other approaches, such as storage offloading [34], attempt to circumvent GPU memory limits via system RAM or SSD. However, they suffer from severe I/O bandwidth bottlenecks, often resulting in higher latencies than pure cloud solutions. Moreover, distributed edge inference [11], [35], which partitions models across multiple devices, is largely undermined by the prohibitive synchronization overheads over bandwidth-limited networks. Ultimately, relying strictly on resource-constrained local environments is inadequate for high-performance LLM inference.

Speculative Decoding. Speculative Decoding has emerged as a promising paradigm to accelerate autoregressive generation while maintaining lossless consistency with target LLMs [4]–[6]. The fundamental speculative decoding process employs a lightweight draft model to rapidly generate candidate sequences for parallel verification by the target model. To improve drafting efficiency and acceptance rates, recent advances have diversified drafting mechanisms, exploring self-speculative decoding [36], retrieval-based methods [37], and draft-free generation architectures [18]. Furthermore, state-of-the-art approaches widely adopt tree-based decoding [16], [20], [38], [39] to explore multiple generation paths simultaneously, maximizing the yield of each verification pass. However, these advanced speculative decoding techniques are inherently designed for cloud-only infrastructures equipped with shared memory and high-bandwidth interconnects, rendering them inapplicable to edge-cloud collaborative scenarios.

Edge-Cloud Collaborative Inference. To alleviate the burden of cloud servers in LLM inference, edge-cloud collaboration has been actively studied. Early collaborative paradigms predominantly relied on split computing [40]. While effective for Convolutional Neural Networks (CNNs), it incurs unacceptable communication overheads for LLMs due to the continuous transmission of massive intermediate activation tensors. To circumvent tensor transmission entirely, collaborative speculative decoding extends the aforementioned speculative decoding paradigm by offloading the draft model to the edge while retaining the target model on the cloud [13], [14]. Nevertheless, it introduces a new communication bottleneck that necessitates the continuous transmission of drafted tokens alongside their high-dimensional probability distributions (logits) over constrained networks [9]. Even with optimized protocols like Top-K Sparse Logits [10], transmitting these heavy payloads remains prohibitive. In stark contrast, our framework pioneers a token-only communication protocol to fully eradicating this limitation. In stark contrast, our framework pioneers a token-only communication protocol to overcome this bottleneck and achieve full acceleration.

VI. CONCLUSION

In this paper, we proposed Ygg, an efficient edge-cloud collaborative speculative decoding framework with minor transmission overhead to improve LLM inference QoS. By offloading the lightweight draft model to cost-effective edge devices and employing a token-only transmission mechanism, Ygg successfully resolves both the severe GPU memory constraints of cloud-only setups and the heavy communication bottlenecks found in existing collaborative methods. In the framework, we introduced two key mechanisms: a budget-aware global adaptive tree-based draft generation scheme that ensures high token acceptance rates with minimal overhead, and an asynchronous pipelining strategy that overlaps edge drafting with cloud verification to eliminate idle computational bubbles. Extensive experiments verify that it can effectively improve throughput and reduce cost of LLM inference.

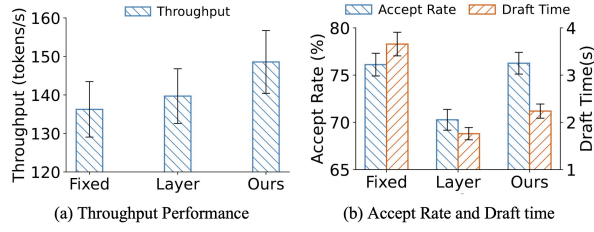


图 10: 树构建算法的影响。

扩展的预算同时增加了单次交互轮次（起草、验证和通信）的计算和传输延迟。因此，如图9(a)所示，*the throughput exhibits a concave trend: it initially increases as the benefits of a higher acceptance rate dominate, but eventually decreases when the per-round latency overhead outweighs the reduction in interaction rounds.*

树构建算法的影响。图 10 将我们的预算感知全局自适应树构建算法与标准的固定和按层扩展算法进行了比较。*Fixed* 策略在剪枝前大胆地扩展一个大型的规则树；虽然这种穷尽式的方法可以获得最高的令牌接受率，但其吞吐量受到边缘设备上巨量冗余草稿时间开销的瓶颈限制。相反，*Layer-wise* 方法的接受率明显较低，因为它严格限制节点选择在当前层内，频繁丢弃位于较浅深度的全局最优节点。我们提出的全局自适应算法有效地解决了这一困境。通过从全局缓冲中动态选择高置信度节点，它在保持可接受计算冗余水平的同时，维持了高度竞争的接受率。因此，它实现了更好的权衡，并在三种策略中提供了最高的整体吞吐量。

五、相关工作

高效的设备端推理。为了减轻大模型在集中式云部署中的巨大开销，近期研究转向了支持本地推理。然而，现有的权宜之计无法在不做重大妥协的情况下解决这一问题。具体而言，权重量化方法 [30], [31] 成功压缩了内存占用，但通常会带来生成质量的不可逆下降。另一种方法是用边缘原生轻量模型替代它们以实现本地执行 [32], [33]，但其推理能力远不及大规模模型。其他方法，如存储卸载 [34]，尝试通过系统 RAM 或 SSD 绕过 GPU 内存限制。但它们受到严重的 I/O 带宽瓶颈影响，往往导致延迟高于纯云解决方案。此外，分布式边缘推理 [11], [35] 将模型分布在多个设备上，但在很大程度上受到障碍的影响。

投机性解码。投机性解码已经成为一种有前景的范式，用于加速自回归生成，同时保持与目标大语言模型的无损一致性 [4]–[6]。基本的投机性解码过程使用轻量级的草稿模型快速生成候选序列，以便目标模型进行并行验证。为了提高草稿效率和接受率，最近的进展多样化了草稿机制，探索了自投机解码 [36]、基于检索的方法 [37] 以及无草稿生成架构 [18]。此外，最先进的方法广泛采用基于树的解码 [16]、[20]、[38]、[39]，以同时探索多条生成路径，最大化每次验证的产出。然而，这些先进的投机性解码技术本质上是仅为云端基础设施设计的，这类基础设施配备了共享内存和高带宽互连，使其无法应用于边缘-云协同环境。

边缘云协同推断。为了缓解云服务器在LLM推断中的堆积，边缘云协作已被积极研究。早期协作范式主要依赖分流计算[40]。虽然对卷积神经网络（CNN）有效，但由于持续传输大量中间作用张量，LLMs会带来不可接受的通信开销。为了完全规避张量传输，协同推测解码通过将草稿模型卸载到边缘，同时保留目标模型在云端，扩展了上述推测解码范式[13], [14]。然而，它引入了一个新的通信瓶颈，要求在受限网络中持续传输已草拟的令牌及其高维概率分布（logits）[9]。即使采用了像Top-K稀疏Logit这样的优化协议[10]，传输这些重负载依然难以实现。

六、结论

在本文中，我们提出了Ygg，一种高效的边缘-云协作推测解码框架，具有较小的传输开销，以提升大型语言模型（LLM）推理的服务质量。通过将轻量级草稿模型卸载到低成本的边缘设备，并采用仅传输令牌的机制，Ygg成功解决了纯云端设置中严重的GPU内存限制问题以及现有协作方法中存在的重通信瓶颈。在该框架中，我们引入了两个关键机制：一种预算感知的全局自适应树状草稿生成方案，能够在最小开销下保证高令牌接受率；以及一种异步流水线策略，实现边缘草稿与云端验证的重叠，从而消除计算空闲气泡。大量实验验证了其能够有效提高吞吐量并降低LLM推理成本。

REFERENCES

- [1] DeepSeek-AI, “DeepSeek-V3 technical report,” DeepSeek-AI, Tech. Rep., 2024, arXiv preprint arXiv:2412.19437.
- [2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [3] R. Pope, S. Douglas, A. Chowdhery *et al.*, “Efficiently scaling transformer inference,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 5, 2023.
- [4] H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei, and Z. Sui, “Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation,” in *Findings of the Association for Computational Linguistics: EMNLP*, 2023, pp. 3909–3925.
- [5] C. Chen, S. Borgeaud, G. Irving *et al.*, “Accelerating large language model decoding with speculative sampling,” *arXiv preprint arXiv:2302.01318*, 2023.
- [6] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proceedings of the International Conference on Machine Learning (ICML 2023)*. PMLR, 2023, pp. 19 274–19 286.
- [7] P. SK, S. A. Kesanapalli, and Y. Simmhan, “Characterizing the performance of accelerated jetson edge devices for training deep learning models,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 6, no. 3, pp. 1–26, 2022.
- [8] P. Nikolaou, Y. Sazeides, A. Lampropoulos, D. Guilhot, A. Bartoli, G. Papadimitriou, A. Chatzidimitriou, D. Gizopoulos, K. Tsvetoglu, L. Mukhanov *et al.*, “On the evaluation of the total-cost-of-ownership trade-offs in edge vs cloud deployments: A wireless-denial-of-service case study,” *IEEE Transactions on Sustainable Computing*, vol. 7, no. 2, pp. 334–345, 2019.
- [9] J. Ning, C. Zheng, and T. Yang, “Dssd: Efficient edge-device llm deployment and collaborative inference via distributed split speculative decoding,” *arXiv preprint arXiv:2507.12000*, 2025.
- [10] C. Zheng and T. Yang, “Communication-efficient collaborative llm inference via distributed speculative decoding,” *arXiv preprint arXiv:2509.04576*, 2025.
- [11] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley *et al.*, “DeepSpeed-Inference: enabling efficient inference of transformer models at unprecedented scale,” in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.
- [12] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, “DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation ai scale,” in *International conference on machine learning*. PMLR, 2022, pp. 18 332–18 346.
- [13] W. Zhao, W. Jing, Z. Lu *et al.*, “Edge and terminal cooperation enabled llm deployment optimization in wireless network,” in *Proceedings of the IEEE/CIC International Conference on Communications in China Workshops (ICCC 2024)*. IEEE, 2024, pp. 220–225.
- [14] Z. Hao, H. Jiang, S. Jiang *et al.*, “Hybrid slm and llm for edge-cloud collaborative inference,” in *Proceedings of the Workshop on Edge and Mobile Foundation Models*, 2024, pp. 36–41.
- [15] X. Zhang, N. Yan, Y. Su, Y. Deng, and T. Mahmoodi, “Communication-aware knowledge distillation for federated llm fine-tuning over wireless networks,” *arXiv preprint arXiv:2509.01750*, 2025.
- [16] X. Miao, G. Oliaro, Z. Zhang *et al.*, “Specinfer: Accelerating large language model serving with tree-based speculative inference and verification,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. ACM, 2024, pp. 932–949.
- [17] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle: Speculative sampling requires rethinking feature uncertainty,” *arXiv preprint arXiv:2401.15077*, 2024.
- [18] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple llm inference acceleration framework with multiple decoding heads,” *arXiv preprint arXiv:2401.10774*, 2024.
- [19] R. Svirschevski, A. May, Z. Chen *et al.*, “Specexec: Massively parallel speculative decoding for interactive llm inference on consumer devices,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 16 342–16 368.
- [20] J. Wang, Y. Su, J. Li *et al.*, “Opt-tree: Speculative decoding with adaptive draft tree structure,” *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 188–199, 2025.
- [21] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, “A review on edge large language models: Design, execution, and applications,” *ACM Computing Surveys*, vol. 57, no. 8, pp. 1–35, 2025.
- [22] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for {Transformer-Based} generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.
- [23] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” *Advances in neural information processing systems*, vol. 32, 2019.
- [24] Google Cloud, “Google cloud pricing,” <https://cloud.google.com/>, 2026, accessed: 2026.
- [25] Vast.ai, “Vast.ai global gpu market pricing,” <https://vast.ai/>, 2026, accessed: 2026.
- [26] L. Jiang, S. D. Fu, Y. Zhu *et al.*, “Janus: Collaborative vision transformer under dynamic network environment,” in *Proceedings of the IEEE Conference on Computer Communications (INFOCOM 2025)*. IEEE, 2025, pp. 1–10.
- [27] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez *et al.*, “Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality,” See <https://vicuna.lmsys.org> (accessed 14 April 2023), vol. 2, no. 3, p. 6, 2023.
- [28] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, “Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in neural information processing systems*, vol. 36, pp. 46 595–46 623, 2023.
- [29] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [30] E. Frantar, S. Ashkboos, T. Hoefler *et al.*, “OPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [31] J. Lin, J. Tang, H. Tang *et al.*, “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 6, 2024.
- [32] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi *et al.*, “MobileLLM: Optimizing sub-billion parameter language models for on-device use cases,” in *Forty-first International Conference on Machine Learning*, 2024.
- [33] P. Zhang, G. Zeng, T. Wang, and W. Lu, “Tinyllama: An open-source small language model,” *arXiv preprint arXiv:2401.02385*, 2024.
- [34] Y. Sheng, L. Zheng, B. Yuan *et al.*, “FlexGen: High-throughput generative inference of large language models with a single GPU,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 31 094–31 116.
- [35] C. Hu and B. Li, “When the edge meets transformers: Distributed inference with transformer models,” in *Proceedings of the 44th IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2024, pp. 82–92.
- [36] J. Zhang, J. Wang, H. Li, L. Shou, K. Chen, G. Chen, and S. Mehrotra, “Draft& verify: Lossless large language model acceleration via self-speculative decoding,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 11 263–11 282.
- [37] Z. He, Z. Zhong, T. Cai, J. Lee, and D. He, “Rest: Retrieval-based speculative decoding,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1)*, 2024, pp. 1582–1595.
- [38] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, “Sequoia: Scalable, robust, and hardware-aware speculative decoding,” *arXiv preprint arXiv:2402.12374*, 2024.
- [39] F. Lin, H. Yi, Y. Yang, H. Li, X. Yu, G. Lu, and R. Xiao, “BitA: Bi-directional tuning for lossless acceleration in large language models,” *Expert Systems with Applications*, vol. 279, p. 127305, 2025.
- [40] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 615–629.

REFERENCES

- [1] DeepSeek-AI, “DeepSeek-V3 技术报告”, DeepSeek-AI, 技术报告, 2024, arXiv 预印本 arXiv:2412.19437. [2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: 开放高效的基础语言模型”, *arXiv preprint arXiv:2302.13971*, 2023. [3] R. Pope, S. Douglas, A. Chowdhery *et al.*, “高效扩展 Transformer 推理”, 发表于 *Proceedings of Machine Learning and Systems (MLSys)*, 第 5 卷, 2023. [4] H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei, 和 Z. Sui, “投机解码: 利用投机执行加速 seq2seq 生成”, 发表于 *Findings of the Association for Computational Linguistics: EMNLP*, 2023, 第 3909–3925 页. [5] C. Chen, S. Borgeaud, G. Irving *et al.*, “通过投机采样加速大型语言模型解码”, *arXiv preprint arXiv:2302.01318*, 2023. [6] Y. Leviathan, M. Kalman, 和 Y. Matias, “通过投机解码快速从 Transformer 推理”, 发表于 *Proceedings of the International Conference on Machine Learning (ICML 2023)*. PMLR, 2023, 第 1927 4–19286 页. [7] P. SK, S. A. Kesanapalli, 和 Y. Simmhan, “表征性能的
- [20] 王杰, 苏勇, 李军 *et al.*, “Opt-tree: 基于自适应草稿树结构的投机解码”, *Transactions of the Association for Computational Linguistics*, 第13卷, 第188–199页, 2025年. [21] 郑毅, 陈勇, 钱斌, 史翔, 舒毅, 陈杰, “边缘大语言模型综述: 设计、执行与应用”, *ACM Computing Surveys*, 第57卷, 第8期, 第1–35页, 2025年. [22] 余国一, 郑在成, 金光伟, 金胜, 秦勃光, “Orca: 基于Transformer生成模型的分布式服务系统,” 载于 *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022年, 第521–538页. [23] Paszke A., Gross S., Massa F., Lerer A., Bradbury J., Chanan G., Killeen T., Lin Z., Gimelshein N., Antiga L. *et al.*, “PyTorch: 一种命令式、高性能深度学习库,” *Advances in neural information processing systems*, 第32卷, 2019年. [24] Google Cloud, “Google云定价,” <https://cloud.google.com/>, 2026年, 访问: 2026年. [25] Vast.ai, “Vast.ai全球GPU市场定价,” <https://vast.ai/>, 2026年, 访问: 2026年. [26] 江磊, 傅思德, 朱勇 *et al.*, “Janus: 协作视觉变换器